

课程笔记

斯坦福 CS336：讲座 7 并行化

五道口纳什 & Codex

2026 年 6 月 8 日



视频作者/频道：脑袋要有光

发布日期：2026 年 5 月 21 日

视频时长：1 小时 21 分 02 秒

视频链接：<https://www.bilibili.com/video/BV12gLx6xEcq?p=7>

目录

1 课程导论与并行层次结构	3
1.1 从单 GPU 到多 GPU 的视角转换	3
1.2 通用层次化存储与通信结构	4
1.3 使用多 GPU 的动机与权衡	5
1.3.1 动机一：显存容量瓶颈	5
1.3.2 动机二：算力扩展需求	6
1.3.3 核心权衡：通信带宽开销	6
1.3.4 讲座结构预告	6
1.4 本章小结	7
2 集体通信原语与硬件互联	8
2.1 集体操作 (Collective Operations) 概览	8
2.2 核心集体操作：All-Gather、Reduce-Scatter、All-Reduce	9
2.2.1 四种基础热身操作	9
2.2.2 全收集 (All-Gather)	12
2.2.3 规约-散播 (Reduce-Scatter)	12
2.2.4 全规约 (All-Reduce)	13
2.3 All-to-All 与操作记忆技巧	14
2.3.1 All-to-All 操作	14
2.3.2 术语记忆技巧	15
2.4 本章小结	16
3 硬件互联架构与 NCCL	17
3.1 从 PCIe 到 NVLink/InfiniBand 的拓扑演进	17
3.1.1 经典 PCIe 拓扑：单节点时代的局限	17
3.1.2 现代数据中心拓扑：NVLink + NVSwitch + InfiniBand	18
3.1.3 带宽层级的逐级下降	19
3.2 RDMA 与大规模集群扩展	20
3.2.1 传统以太网的数据路径与 CPU 瓶颈	20
3.2.2 RDMA：绕过 CPU 的直接内存访问	20
3.2.3 RoCE：以太网上的 RDMA 替代方案	20
3.3 NVL72 与 NCCL 通信库	21
3.3.1 NVL72：打破“八卡节点”的边界	21
3.3.2 NCCL：硬件拓扑之上的集体通信抽象	21
3.3.3 非对称 GPU 配置的通信困境	23
3.4 本章小结	23
4 PyTorch 分布式编程实践	24
4.1 torch.distributed 基础与进程管理	24
4.1.1 动机：从理论原语到可执行代码	24
4.1.2 核心概念：Rank、World Size 与后端选择	24
4.1.3 进程启动：spawn 函数	25

4.1.4	初始化: <code>init_process_group</code>	26
4.1.5	同步屏障: <code>barrier</code> 的作用与开销	27
4.2	All-Reduce、Reduce-Scatter 与 All-Gather 代码实现	28
4.2.1	All-Reduce: 就地求和并广播	28
4.2.2	Reduce-Scatter: 输入输出分离的规约散播	29
4.2.3	All-Gather: 收集分散结果	30
4.2.4	核心等价关系的代码验证	31
4.3	异步操作与有效带宽基准测试	31
4.3.1	异步集体通信: 计算与通信重叠	31
4.3.2	基准测试方法: 预热、计时与同步	33
4.3.3	有效带宽的计算	34
4.3.4	Reduce-Scatter 的带宽计算	36
4.3.5	All-Reduce 与 Reduce-Scatter 的带宽对比	37
4.4	本章小结	37
5	三大并行训练策略	38
5.1	数据并行 (DDP): 分数据、全参数	38
5.1.1	动机与核心思想	38
5.1.2	机制: 局部批次与梯度同步	39
5.1.3	DDP 的优雅性与局限	40
5.2	张量并行 (TP): 分参数、全数据	40
5.2.1	动机与核心思想	40
5.2.2	机制: 列式切分与 All-Gather 通信	40
5.2.3	反向传播的对偶性	42
5.2.4	张量并行的通信代价	43
5.3	流水线并行 (PP): 分层切分与微批次	43
5.3.1	动机与核心思想	43
5.3.2	机制: 点对点通信与微批次	43
5.3.3	微批次: 减少流水线气泡	44
5.3.4	通信与计算重叠	44
5.4	三种并行策略的对比与选择	45
5.4.1	综合对比	45
5.4.2	硬件驱动的策略选择	45
5.4.3	混合并行策略	45
5.5	进阶方向与其他并行类型	46
5.6	本章小结	46
6	总结与延伸	47
6.1	讲座核心脉络回顾	47
6.2	核心洞察与实用建议	48
6.3	前沿方向与开放问题	49
6.4	延伸资源	49

1 课程导论与并行层次结构

本节概览

本节从单 GPU 优化的视角出发，引出多 GPU 并行训练的核心问题。我们将建立一套通用的层次化存储与通信模型，理解从 L1 缓存到 InfiniBand 的带宽递减规律，并分析使用多 GPU 的两大动机及其背后的权衡。

1.1 从单 GPU 到多 GPU 的视角转换

上一讲我们深入探讨了如何通过编写高效的内核 (kernel) 让**单块 GPU** 跑得更快。回顾当时的核心画面：一块 GPU 内部包含高带宽内存 (High Bandwidth Memory, HBM)、L2 缓存、L1 缓存/共享内存 (shared memory)、寄存器 (register) 以及大量流式多处理器 (Streaming Multiprocessor, SM)。优化的关键在于减少数据移动——让算术逻辑单元 (Arithmetic Logic Unit, ALU) 和张量核心 (Tensor Core) 尽可能在靠近计算单元的地方获取所需数据。

本讲将这幅画面进行了扩展：我们不再只关注一个 GPU 盒子，而是面对四块、八块乃至上千块 GPU 组成的计算集群，如图 1 所示。每块 GPU 仍然拥有自己的 SM、L2 缓存和 HBM，但 GPU 之间需要通过高速互联链路交换数据。

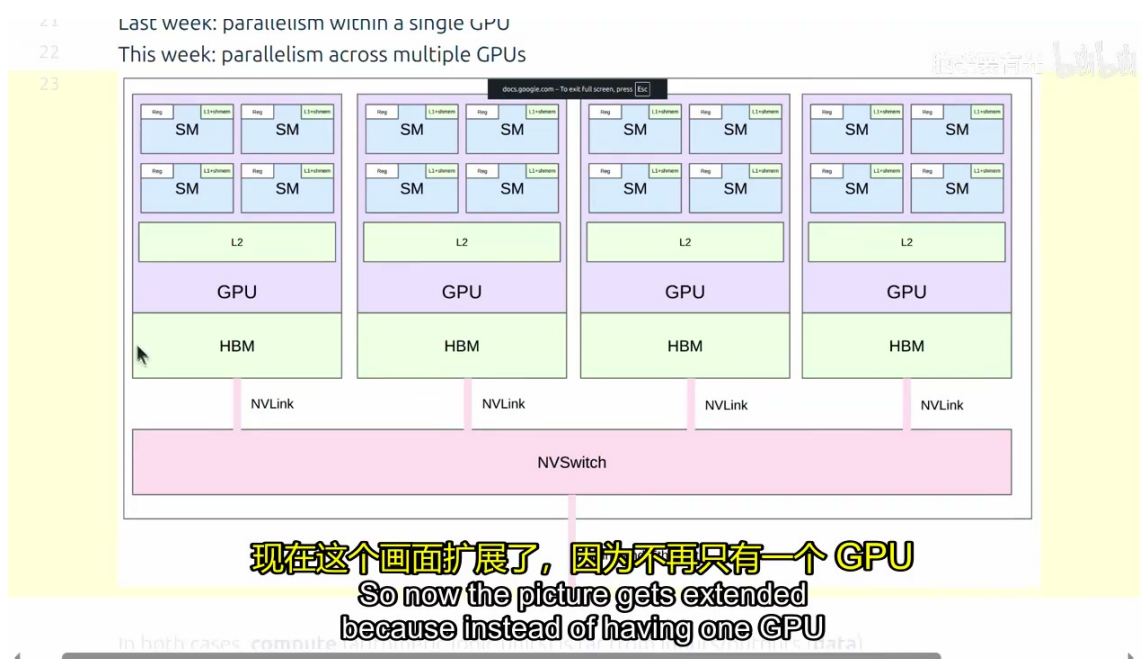


图 1: 从单 GPU 内部并行扩展到多 GPU 集群并行。每块 GPU 包含 SM、L2 缓存和 HBM，GPU 之间通过 NVLink 和 NVSwitch 互联，跨节点则通过 InfiniBand 或以太网连接。

核心洞察：统一的优化主题

无论是单 GPU 还是多 GPU 场景，**计算资源 (ALU、Tensor Core) 离所需数据都很远**。在单 GPU 中，”远”意味着数据存放在 HBM 里；在多 GPU 场景中，”远”意味着数据可能位于另一块 GPU 的 HBM 上。因此，两者的优化原理是相通的——**核心任务都是精心编排计算，尽量避免数据传输瓶颈**。

这一视角转换揭示了一个深刻的设计原则：多 GPU 并行并非引入全新的优化范式，而是将单

GPU 中的”内存层次”概念推广到”分布式内存层次”。在单 GPU 中，我们通过融合 (fusion)、分块 (tiling) 等技术减少 HBM 访问；在多 GPU 中，我们将通过数据复制 (replication) 与分片 (sharding) 来减少跨 GPU 通信量。

1.2 通用层次化存储与通信结构

为了系统地理解多 GPU 系统中的数据移动代价，我们可以建立一个**通用层次化存储与通信结构** (Generalized Hierarchy)。这个层次结构从快到慢、从局部到全局，涵盖了现代 GPU 集群中所有关键的存储与通信层级，如图 2 所示。

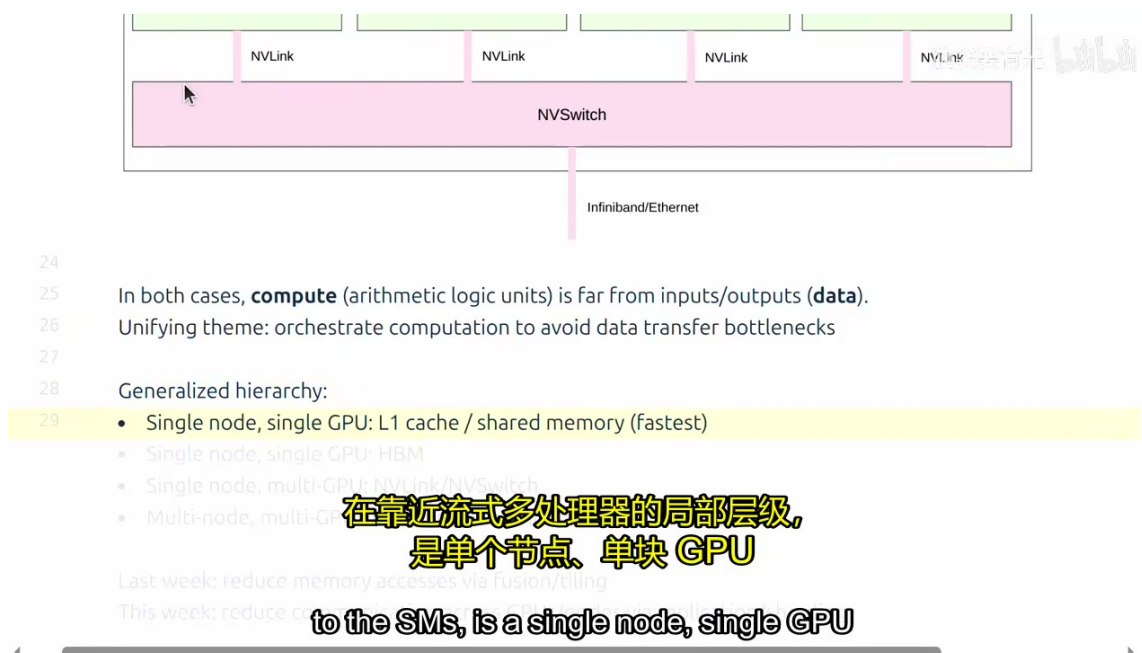


图 2: 通用层次化存储与通信结构。从上到下，存储/通信速度逐级递减：L1 缓存/共享内存（最快）→ HBM → NVLink/NVSwitch → InfiniBand/以太网（最慢）。

具体而言，该层次结构包含以下四个层级：

1. **单节点单 GPU: L1 缓存 / 共享内存**（最快）——位于 SM 附近，带宽可达数十 TB/s 量级，是寄存器之后最快的存储层级。
2. **单节点单 GPU: HBM** ——单 GPU 的全局内存，带宽约为数 TB/s（如 H100 的 HBM 带宽约 3.35 TB/s）。在上一讲中，HBM 是我们努力绕过的”慢速”瓶颈；但在多 GPU 视角下，HBM 反而成了相对快速的本地存储。
3. **单节点多 GPU: NVLink / NVSwitch** ——单节点内 GPU 之间的高速点对点互联。以 NVLink 5 为例，单链路带宽可达 1.8 TB/s，约为 HBM 带宽的 1/4 到 1/2 量级。NVSwitch 可实现节点内多达 72 块 GPU 的全互联 (full connectivity)。
4. **多节点多 GPU: InfiniBand / 以太网**（最慢）——跨节点网络互联。InfiniBand NDR 400 可提供 400 Gb/s（约 50 GB/s）的链路带宽，标准以太网则更慢。这是整个层次中最昂贵的通信路径。

带宽数量级的断崖式下降

从 L1 缓存到跨节点网络，有效带宽可能相差三个数量级以上。这意味着：一次跨节点数据传输的代价，可能相当于数千次本地 HBM 访问。因此，并行策略的设计必须严格遵循“数据局部性优先”原则——尽量让计算在数据所在的位置完成，避免不必要的远程访问。

上周我们学习的单 GPU 优化技巧——融合（fusion）和分块（tiling）——本质上都是通过提升数据复用率来减少慢速存储访问。其典型模式是：

1. 将数据从 HBM 读到共享内存；
2. 在共享内存中完成尽可能多的计算；
3. 最后将结果写回 HBM。

本周的主题可以看作这一思路的自然延伸：**如何通过合理的数据复制与分片（replication / sharding），减少 GPU 之间以及节点之间的通信量**。在多 GPU 场景中，“共享内存”的角色被扩展为“另一块 GPU 上的 HBM”，而优化的目标仍然是：在数据传输代价最低的地方完成尽可能多的计算。

1.3 使用多 GPU 的动机与权衡

既然使用多 GPU 会引入复杂的通信问题，为什么我们还必须走向分布式训练？讲座中给出了两个根本性的动机，如图 3 所示。

25 In both cases, **compute** (arithmetic logic units) is far from inputs/outputs (**data**).

26 Unifying theme: orchestrate computation to avoid data transfer bottlenecks

27

28 Generalized hierarchy:

29 • Single node, single GPU: L1 cache / shared memory (fastest)

30 • Single node, single GPU: HBM

31 • Single node, multi-GPU: NVLink/NVSwitch

32 • Multi-node, multi-GPU: Infiniband/Ethernet (slowest)

33

34 Last week: reduce memory accesses via fusion/tiling

35 This week: reduce communication across GPUs/nodes via replication/sharding

Why do multi-GPU?

1. Your parameters (optimizer state + gradients + activations) don't fit on a single GPU.

2. You want to use more GPUs (more FLOPs) to train faster.

GPU 间的通信量

sharding appropriately.

图 3: 使用多 GPU 的两大动机：（1）模型参数、梯度与激活值超出单卡显存容量；（2）利用更多 GPU 的算力加速训练。

1.3.1 动机一：显存容量瓶颈

第一个也是最直接的动机是**显存容量不足**。现代大语言模型的训练不仅需要存储模型参数，还需要保存梯度（gradients）、优化器状态（optimizer state，如 Adam 中的动量和二阶矩估计）以及

前向传播中的激活值 (activations)。这些内容的总和往往远超单块 GPU 的显存容量。

以 NVIDIA B200 为例, 其 HBM 显存容量为 192 GB。然而, 一个 1 万亿参数 (1T parameters) 的模型, 即使以 BF16 格式存储参数也需要约 2 TB 显存, 再加上优化器状态和激活值, 总需求可能达到数 TB 量级。**单块 GPU 根本放不下这么大的模型**, 因此必须将模型状态分片到多块 GPU 上。

1.3.2 动机二: 算力扩展需求

第二个动机是**加速训练**。即使模型能够装进单块 GPU, 训练一个大模型可能需要数月甚至数年时间。为了缩短训练周期, 研究者希望利用更多 GPU 的峰值算力 (FLOPs) 来并行处理。将模型或数据拆分到多张 GPU 上, 可以让更多计算核心同时工作, 从而缩短 wall-clock 时间。

1.3.3 核心权衡: 通信带宽开销

然而, 更多 GPU 并不意味着线性加速。这里存在一个关键的**权衡** (trade-off):

- 如果你把所有参数都塞进尽可能少的 GPU 里, 虽然通信最少, 但**能利用的计算核心数量就会减少**, 训练时间变长。
- 如果你把模型分散到更多 GPU 上, 虽然算力增加了, 但**必须承受额外的通信带宽开销**——GPU 之间需要频繁同步梯度、激活值或参数分片。

并行策略选择的核心公式

理想情况下, 分布式训练的加速比受限于计算与通信的比例。设单次迭代中计算时间为 T_{compute} , 通信时间为 T_{comm} , 则有效加速比大致为:

$$\text{效率} \approx \frac{T_{\text{compute}}}{T_{\text{compute}} + T_{\text{comm}}}$$

当 $T_{\text{comm}} \ll T_{\text{compute}}$ 时, 接近线性加速; 当通信与计算相当时, 扩展效率急剧下降。因此, 选择并行策略本质上是在**最小化通信量与最大化计算并行度**之间寻找最优平衡点。

这意味着你需要”好好算一笔账”——根据模型大小、批次大小、网络拓扑和硬件带宽, 决定采用何种并行策略 (数据并行、张量并行、流水线并行或其组合)。本讲座后续章节将系统介绍这些策略的原理与实现。

1.3.4 讲座结构预告

本讲座分为两大部分, 如图 4所示:

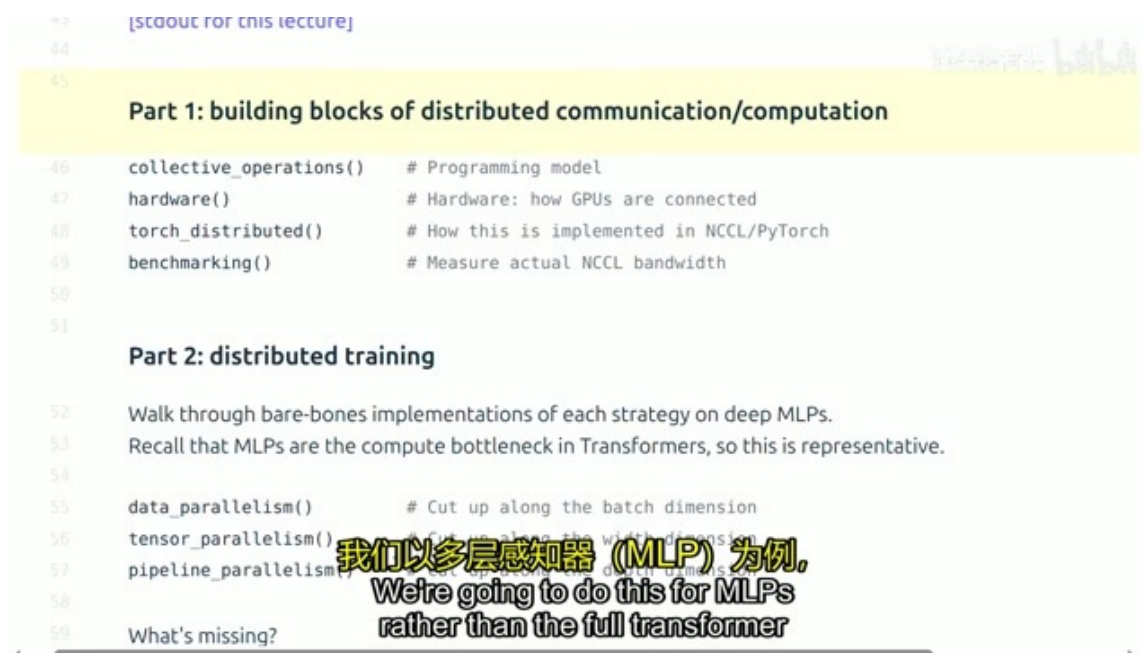


图 4: 讲座整体结构: 第一部分聚焦分布式通信与计算的基础构建块 (集体通信、硬件互联、PyTorch 实现、带宽基准测试); 第二部分探讨三种并行训练策略 (数据并行、张量并行、流水线并行)。

• 第一部分：分布式通信与计算的基础构建块

- 集体通信操作 (Collective Operations) —— 编程模型
- 硬件互联架构 (Hardware) —— GPU 如何物理连接
- torch.distributed —— NCCL/PyTorch 中的实现
- 基准测试 (Benchmarking) —— 测量实际 NCCL 带宽

• 第二部分：分布式训练策略

- 数据并行 (Data Parallelism) —— 沿批次维度切分
- 张量并行 (Tensor Parallelism) —— 沿宽度维度切分
- 流水线并行 (Pipeline Parallelism) —— 沿深度维度切分

讲座将以多层感知器 (Multi-Layer Perceptron, MLP) 为例演示三种并行策略, 而非完整的 Transformer 模型。这是因为 MLP 中的矩阵乘法是 Transformer 中的计算瓶颈, 具有充分的代表性。

代码执行说明

本讲座的所有代码均用 Python 编写, 可直接运行。当直接执行讲座脚本时 (python lecture_07.py), 代码默认使用多进程 (multiprocessing) 模式。但在讲座的逐步讲解过程中, 讲师会切换到特殊的单进程模式以方便演示。如需查看多进程模式下的标准输出, 可参考讲座页面中的对应按钮。

1.4 本章小结

本节建立了多 GPU 并行训练的宏观视角, 核心要点如下:

1. **视角转换**: 从单 GPU 优化扩展到多 GPU 并行, 核心挑战不变——计算单元离数据很远, 需要精心编排计算以减少数据传输瓶颈。
2. **通用层次结构**: 现代 GPU 集群呈现四层存储/通信层次: L1/共享内存 → HBM → NVLink/NVSwitch → InfiniBand/以太网, 带宽逐级下降可达三个数量级。
3. **优化思路的延续**: 单 GPU 中的融合与分块技巧, 对应到多 GPU 场景即为数据复制与分片, 目标都是减少慢速层级的数据移动。
4. **使用多 GPU 的两大动机**: (1) 显存容量不足——模型状态无法装入单卡; (2) 算力扩展需求——利用更多 FLOPs 加速训练。
5. **关键权衡**: 更多 GPU 带来更高并行度, 但也引入额外通信开销。并行策略的选择需要在计算效率与通信代价之间精打细算。

在后续章节中, 我们将首先深入理解分布式通信的基本原语——**集体通信操作** (Collective Operations), 然后考察硬件互联拓扑与 NCCL 通信库, 最后在 PyTorch 中动手实现并测量实际带宽, 为理解三种并行训练策略奠定坚实的基础。

2 集体通信原语与硬件互联

2.1 集体操作 (Collective Operations) 概览

动机: 当我们把训练任务扩展到多块 GPU 时, 数据不再局限于单张卡的显存之内, 计算结果也分散在不同设备上。如果程序员需要手动编写每一对 GPU 之间的点对点 (point-to-point) 通信代码, 不仅工作量巨大, 而且极易出错、难以优化。因此, 分布式编程领域从上世纪 80 年代起便抽象出一组**集体通信原语** (Collective Communication Primitives), 让开发者只需描述“做什么”, 而由底层通信库自动决定“怎么做”。

集

体操作 (Collective Operations) 是分布式编程中的基本原语, 其历史可追溯到 80 年代的并行计算文献。时至今日, 这些底层通信语义仍是大规模语言模型训练的核心基础设施。

核心思想: 所谓“集体” (Collective), 指的是你只需定义一种跨多个设备的通用通信模式或模板, 而不必手动管理某块 GPU 与另一块 GPU 之间的点对点通信。系统会自动帮你处理路由、缓冲、同步等大量细节, 从而使并行编程更加可靠、高效。

基本设定: 在集体通信的语境中, 我们使用以下标准术语:

- **进程** (Process): 每个进程对应一个特定的计算设备, 通常是一块 GPU 或 TPU。
- **Rank**: 进程的编号, 从 0 开始递增, 用于唯一标识每个参与通信的进程。
- **World Size** (N): 参与通信的进程总数, 即设备总数。例如, 若有 4 块 GPU, 则 $N = 4$, Rank 分别为 0, 1, 2, 3。

如图 5 所示, 常见的集体操作包括: 广播 (Broadcast)、散播 (Scatter)、收集 (Gather)、规约 (Reduce)、全收集 (All-Gather)、规约-散播 (Reduce-Scatter)、全规约 (All-Reduce) 以及

⁰截图来源: 讲座幻灯片, 时间戳约 00:06:47。

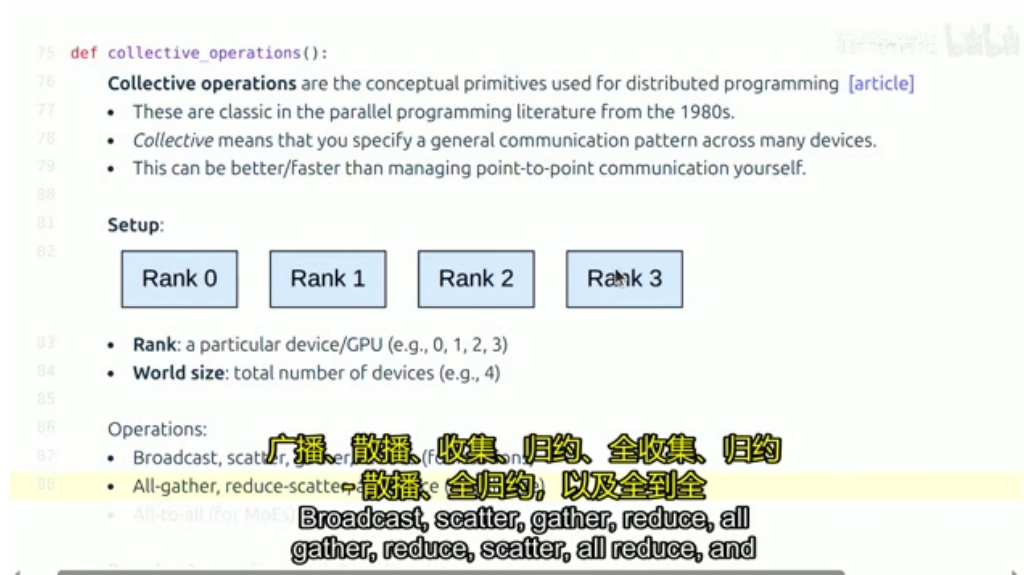


图 5: 集体通信的基本设定：4 个进程（Rank 0–3），World Size $N = 4$ ，以及常见的集体操作列表。

全到全（All-to-All）。其中，前四种（Broadcast、Scatter、Gather、Reduce）被讲师称为“热身操作”——它们虽不是训练流程中最核心的操作，却是理解后续高级原语的关键基石。而 All-Gather、Reduce-Scatter 和 All-Reduce 则是语言模型分布式训练中的“主力军”，会在后续章节反复出现。All-to-All 则主要用于混合专家模型（MoE）的动态路由场景。

2.2 核心集体操作：All-Gather、Reduce-Scatter、All-Reduce

在深入核心操作之前，我们先快速回顾四种基础操作，建立统一的认知框架。

2.2.1 四种基础热身操作

(1) 广播（Broadcast）

广播操作的动机很简单：在分布式训练开始时，通常只有一个进程（如 Rank 0）加载了初始检查点或超参数配置，需要将这些数据复制到所有其他进程，以确保各设备从同一份状态出发。

如图 6 所示，输入为 Rank 0 上的张量 $\text{tensor}([0, 1, 2, 3])$ ；操作结束后，Rank 0–3 均持有 $\text{tensor}([0, 1, 2, 3])$ 。广播在训练核心循环中并不常见，通常只用于初始化阶段一次性执行。

(2) 散播（Scatter）

散播是广播的“反向”思路：不是复制整份数据，而是将根进程（Rank 0）上的大张量均匀切分成与 World Size 相等的份数，再分发给各个进程。

如图 7 所示，输入为 Rank 0 上的 $\text{tensor}([0, 1, 2, 3])$ ；输出为 Rank 0 得到 $[0]$ ，Rank 1 得到 $[1]$ ，Rank 2 得到 $[2]$ ，Rank 3 得到 $[3]$ 。散播操作本身在训练中较少直接使用，但它是理解 Reduce-Scatter 的关键基础：其本质是把一个大张量从一个地方分发到多个不同的地方，使每块 GPU 能对各自分到的部分进行本地计算。

(3) 收集（Gather）

收集是散播的逆操作：输入是分散在各个进程上的多块数据，输出是将这些数据块按顺序拼接后汇总到根进程（Rank 0）。

⁰截图来源：讲座幻灯片，时间戳约 00:08:23。

⁰截图来源：讲座幻灯片，时间戳约 00:09:12。

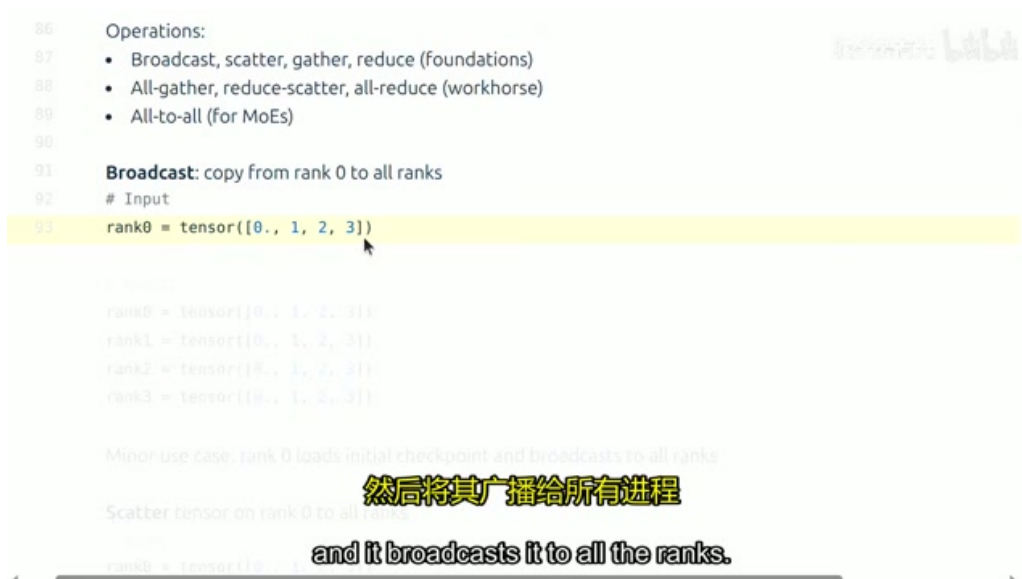


图 6: Broadcast 操作: Rank 0 上的张量 [0, 1, 2, 3] 被复制到所有 Rank, 每个进程最终持有相同的数据。

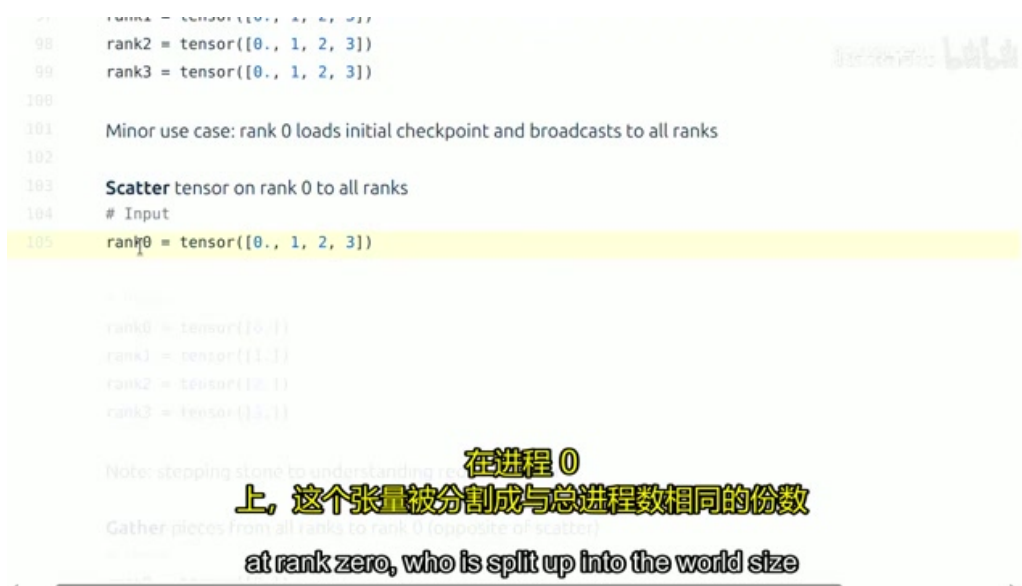


图 7: Scatter 操作: Rank 0 上的张量 [0, 1, 2, 3] 被切分为 4 份, 分别散播到 Rank 0-3。

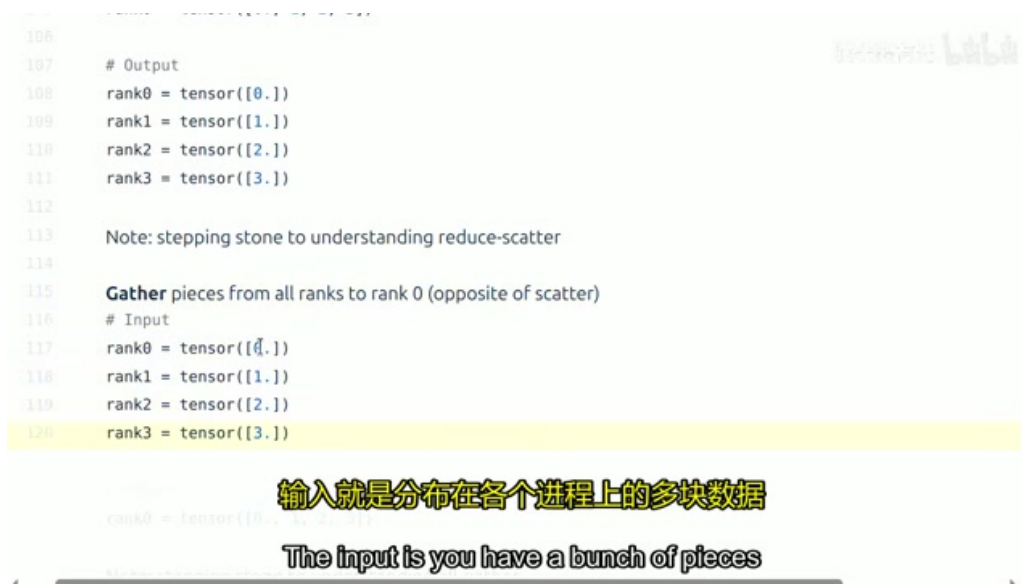


图 8: Gather 操作：将 Rank 0-3 上的数据块 [0],[1],[2],[3] 收集到 Rank 0，拼接为 [0,1,2,3]。

如图 8所示，Gather 的输入是各 Rank 上的独立数据块，输出仅在 Rank 0 上产生完整的拼接张量。与 Scatter 类似，Gather 本身也不是训练循环的核心操作，但它是理解 All-Gather 的必经之路。

(4) 规约 (Reduce)

如果你熟悉函数式编程中的 reduce 操作，这里的规约概念完全一致：对所有进程上的数据执行某种可结合、可交换的运算（如求和、取最大值、取最小值），并将最终结果汇总到根进程。

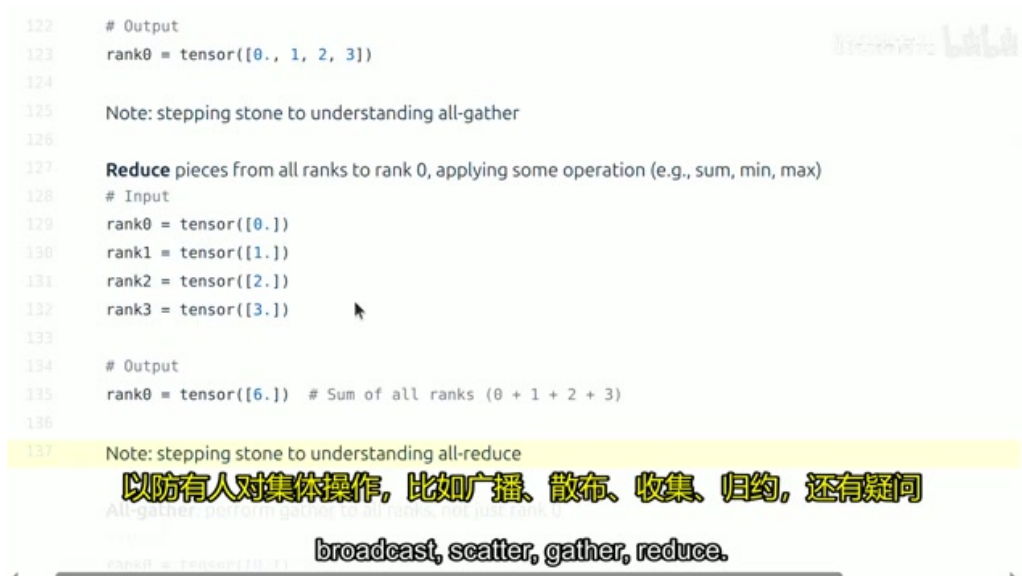


图 9: Reduce 操作：对 Rank 0-3 上的数据 [0],[1],[2],[3] 执行求和规约，结果 [6] 汇总到 Rank 0。

如图 9所示，输入为各 Rank 上的标量 0,1,2,3；执行求和规约后，Rank 0 得到结果 6。可以把 Gather 看作是一种特殊的 Reduce，只不过其“运算”是拼接而非数值计算。理解 Reduce 是理解 All-Reduce 的基础。

⁰截图来源：讲座幻灯片，时间戳约 00:10:13。

⁰截图来源：讲座幻灯片，时间戳约 00:10:47。

catter 与 Gather 互为逆操作；Reduce 是可结合、可交换的组合运算（sum, min, max）。所有基础操作的共同点是：结果最终只面向单个目标进程（通常是 Rank 0）。

2.2.2 全收集（All-Gather）

动机：在模型并行或参数分片场景中，每个进程只保存部分参数（parameter shard）。但在前向传播时，通常需要完整的参数才能进行计算。此时，我们需要一种操作，让每个进程都能获得全部参数的完整副本。

机制：All-Gather 本质上就是“对所有进程都执行一次 Gather”。Gather 操作把分散的数据集中到 Rank 0；All-Gather 则把结果复制到每一个 Rank 上。

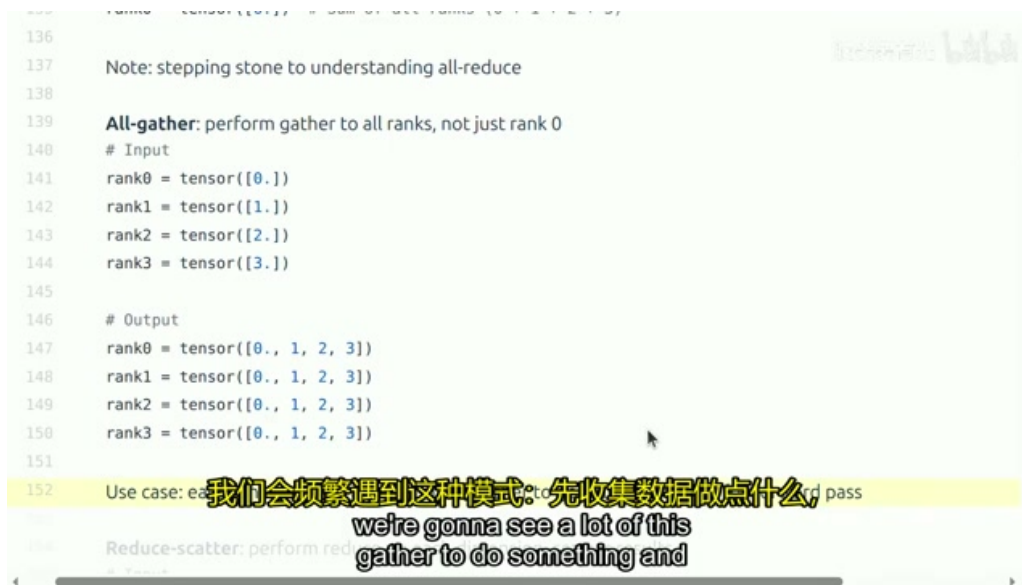


图 10: All-Gather 操作：每个 Rank 最初持有一个数据块，操作后所有 Rank 均获得完整的拼接张量 [0, 1, 2, 3]。

如图 10 所示，输入为 Rank 0-3 分别持有 [0], [1], [2], [3]；All-Gather 的输出是每个 Rank 都持有完整的 [0, 1, 2, 3]。这里的“全”（All）字含义是：对所有进程都输出结果。在后续的张量并行（Tensor Parallelism）中，All-Gather 被用来汇总各进程计算的局部激活值，以得到完整的前向传播结果。

2.2.3 规约-散播（Reduce-Scatter）

动机：在反向传播后，每个 GPU 处理的是不同的数据分片，因此计算出的梯度也分散在不同设备上。我们需要把这些来自不同分片的梯度汇总求和，但又不能把所有结果都堆到某一个进程上——否则会造成显存瓶颈。Reduce-Scatter 正是为了解决这一矛盾：先规约，再把规约结果分散到各个进程。

机制：假设有 4 块 GPU，每块上都有一个 4 维向量。Reduce-Scatter 要求对张量的每个分量分别做一个规约，再把结果放到不同的 GPU 上。

⁰截图来源：讲座幻灯片，时间戳约 00:12:38。

⁰截图来源：讲座幻灯片，时间戳约 00:13:45。

```

148 rank1 = tensor([0., 1, 2, 3])
149 rank2 = tensor([0., 1, 2, 3])
150 rank3 = tensor([0., 1, 2, 3])
151
152 Use case: each rank holds parameter shard, gather to get full parameters for forward pass
153
154 Reduce-scatter: perform reduce on each dimension, scatter results
155 # Input
156 rank0 = tensor([0., 1, 2, 3])
157 rank1 = tensor([1., 2, 3, 4])
158 rank2 = tensor([2., 3, 4, 5])
159 rank3 = tensor([3., 4, 5, 6])
160
161 # Output
162 rank0 = tensor([6.]) # Sum along dim 0 (0 + 1 + 2 + 3)
163 rank1 = tensor([10.]) # Sum along dim 1 (1 + 2 + 3 + 4)
164 rank2 = tensor([14.]) # Sum along dim 2 (2 + 3 + 4 + 5)
165 rank3 = tensor([18.]) # Sum along dim 3 (3 + 4 + 5 + 6)
166
167 Use case: after backward pass, gather gradients, but distribute storage

```

再把结果放到不同的 GPU 上。先看第一个维度
and then I'm going to put it on a different So the first

图 11: Reduce-Scatter 操作：对每块 GPU 上的 4 维向量逐维度求和，再将规约结果分散到不同 Rank。

如图 11所示，输入为：

Rank 0 : [0, 1, 2, 3]

Rank 1 : [1, 2, 3, 4]

Rank 2 : [2, 3, 4, 5]

Rank 3 : [3, 4, 5, 6]

对第 0 个维度求和 ($0 + 1 + 2 + 3 = 6$) 放到 Rank 0，第 1 个维度求和 ($1 + 2 + 3 + 4 = 10$) 放到 Rank 1，第 2 个维度求和 ($2 + 3 + 4 + 5 = 14$) 放到 Rank 2，第 3 个维度求和 ($3 + 4 + 5 + 6 = 18$) 放到 Rank 3。最终输出为：

Rank 0 : [6], Rank 1 : [10], Rank 2 : [14], Rank 3 : [18]

R

Reduce-Scatter 的典型用例：反向传播后，将来自不同数据分片的所有梯度汇总求和，但把求和后的结果分散存储到各个设备上，避免单点显存爆炸。

2.2.4 全规约 (All-Reduce)

动机：数据并行 (Data Parallelism) 是分布式训练中最基础的并行策略。其核心流程是：每个 GPU 用不同的数据子集计算梯度，然后需要把所有梯度求和，再让每块 GPU 都拿到完整的梯度总和，以便同步更新参数。All-Reduce 正是完成“求和 + 广播”这一组合动作的原语。

机制：如果你已经理解了 Reduce-Scatter 和 All-Gather，那么 All-Reduce 就是两者最简单的组合：

$$\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather} \quad (1)$$

先用 Reduce-Scatter 对每个维度做规约并把结果分散到各进程，再用 All-Gather 把这些分散的规约结果收集并广播到所有进程。

⁰截图来源：讲座幻灯片，时间戳约 00:15:21。

```
168
169 All-reduce = reduce-scatter + all-gather
170 # Input
171 rank0 = tensor([0., 1, 2, 3])
172 rank1 = tensor([1., 2, 3, 4])
173 rank2 = tensor([2., 3, 4, 5])
174 rank3 = tensor([3., 4, 5, 6])
175
176 # Output
177 rank0 = tensor([6., 10, 14, 18])
178 rank1 = tensor([6., 10, 14, 18])
179 rank2 = tensor([6., 10, 14, 18])
180 rank3 = tensor([6., 10, 14, 18])
181
182 Use case: after backward pass, sum gradients from different data shards, but replicate full parameters
183 Breaking all-reduce into reduce-scatter + all-gather allows for flexibility (e.g., ZeRO/FSDP)
184
185 All-to-all: each rank sends each other rank some tensor (most general)
186 # Input
187 rank0 = tensor([0., 1, 2, 3]) # send 0 to rank 0, 1 to rank 1, 2 to rank 2, 3 to rank 3
```

图 12: All-Reduce 操作：等价于 Reduce-Scatter 后接 All-Gather，最终每个 Rank 都持有完整的规约结果 [6, 10, 14, 18]。

如图 12 所示，输入与 Reduce-Scatter 相同；经过 All-Reduce 后，每个 Rank 都持有完整的求和结果 [6, 10, 14, 18]。在数据并行训练中，All-Reduce 用于在反向传播后对梯度求和（有时再除以 World Size 求平均），确保所有进程的参数保持一致更新。

虽

然 All-Reduce 在概念上可以看作 Reduce-Scatter + All-Gather，但在实际通信库（如 NCCL）中，All-Reduce 通常有专门优化的实现，不一定真的拆成两步执行。然而，在更高级的技术（如 ZeRO/FSDP）中，显式地把 All-Reduce 拆成 Reduce-Scatter 和 All-Gather 两个独立步骤，可以在中间插入额外的优化逻辑（如梯度压缩、重叠计算），从而获得更大的灵活性。

2.3 All-to-All 与操作记忆技巧

2.3.1 All-to-All 操作

动机：混合专家模型（Mixture of Experts, MoE）的核心思想是动态路由机制——根据当前输入数据，决定将激活值路由到哪些专家网络进行处理。由于每个进程既拥有数据的一部分，也拥有专家网络的一个子集，因此需要一种最通用的通信模式：每个进程可以自定义向其他任意进程发送任意数量的数据。

机制：All-to-All 是最通用的集体通信原语。你需要指定每个进程应该向另一个进程发送什么样的消息。在均衡分配的特殊情况下，All-to-All 等价于一个矩阵转置操作。

⁰截图来源：讲座幻灯片，时间戳约 00:16:50。

```
184
185 All-to-all: each rank sends each other rank some tensor (most general)
186 # Input
187 rank0 = tensor([0., 1, 2, 3]) # send 0 to rank 0, 1 to rank 1, 2 to rank 2, 3 to rank 3
188 rank1 = tensor([4., 5, 6, 7]) # send 4 to rank 0, 5 to rank 1, 6 to rank 2, 7 to rank 3
189 rank2 = tensor([8., 9, 10, 11]) # send 8 to rank 0, 9 to rank 1, 10 to rank 2, 11 to rank 3
190 rank3 = tensor([12., 13, 14, 15]) # send 12 to rank 0, 13 to rank 1, 14 to rank 2, 15 to rank 3
191
192 # Output
193 rank0 = tensor([0, 4, 8, 12])
194 rank1 = tensor([1, 5, 9, 13])
195 rank2 = tensor([2, 6, 10, 14])
196 rank3 = tensor([3, 7, 11, 15])
197
198 Notes:
199 • Useful for ...
200 • For balanced splits, all-to-all looks like ...
201 • Also handles unbalanced ...
```

你会发现，第一个进程会收到所有那些被放在第 0 列的数据
rank is going to receive everyone who sent everything in column zero

图 13: All-to-All 操作：每个 Rank 按列发送数据，最终每个 Rank 收到对应列的全部数据，均衡情况下等价于矩阵转置。

如图 13 所示，输入为 4 个 Rank 各自持有的 4 维向量：

Rank 0 : [0, 1, 2, 3] (0 发给 Rank 0, 1 发给 Rank 1, 2 发给 Rank 2, 3 发给 Rank 3)

Rank 1 : [4, 5, 6, 7] (4 发给 Rank 0, 5 发给 Rank 1, 6 发给 Rank 2, 7 发给 Rank 3)

Rank 2 : [8, 9, 10, 11]

Rank 3 : [12, 13, 14, 15]

输出时，每个 Rank 收到所有进程中对应该列位置的数据：

Rank 0 : [0, 4, 8, 12] (所有进程的第 0 列)

Rank 1 : [1, 5, 9, 13] (所有进程的第 1 列)

Rank 2 : [2, 6, 10, 14]

Rank 3 : [3, 7, 11, 15]

如果将通信模式看作一个矩阵，那么均衡 All-to-All 实质上就是对该矩阵做转置。当然，All-to-All 也能处理数据分配不均衡的情况——你可以配置该操作向任意进程发送任意数量的字节。但在实际训练中，我们通常希望数据分配尽可能均衡（如 MoE 中的负载均衡），以避免某些进程成为通信瓶颈。

2.3.2 术语记忆技巧

面对如此多的集体操作名称，讲师在课程末尾提供了几条简洁的记忆法则：

集体通信术语记忆技巧：

1. **Reduce**：执行某种可结合、可交换的运算（sum, min, max），将多份数据“归约”为一份结果。
2. **Scatter** 与 **Gather** 互为逆操作：Scatter 是把一份大的分散成多份小的；Gather 是把多份小的集中成一份大的。
3. **All**-前缀的含义：目标面向所有设备。All-Gather = 对所有 Rank 做 Gather；All-Reduce = 规约后广播给所有 Rank。

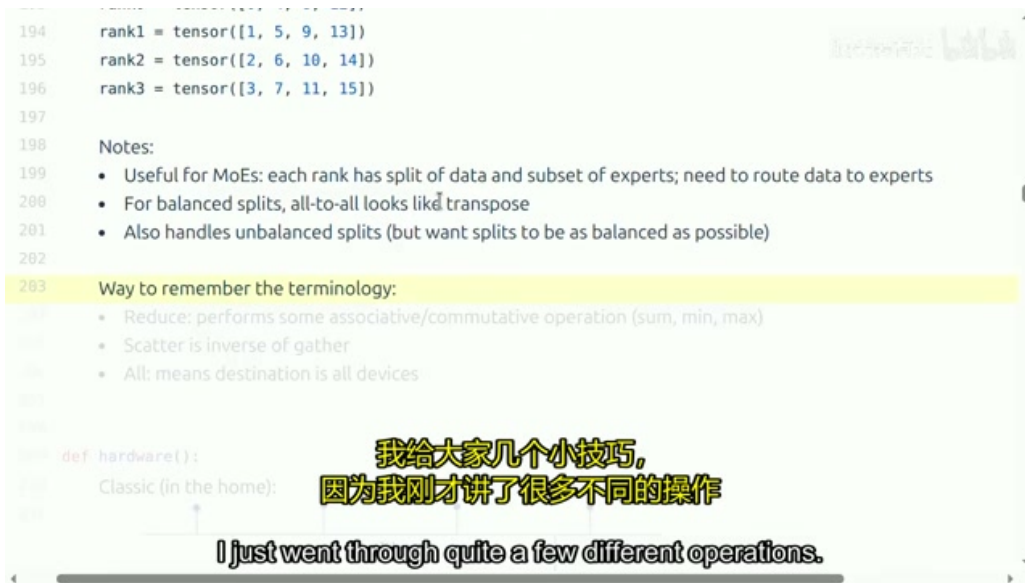


图 14: 讲座中总结的术语记忆要点：Reduce 是可交换组合运算；Scatter 是 Gather 的逆操作；All 表示目标为所有设备。

此外，关于 Rank 0 的特殊性：在 Gather 和 Reduce 的示例中，我们以 Rank 0 作为目标进程，但这只是举例。在实际调用 API 时，你可以指定任意 Rank 作为输出目标——目标进程是在调用操作时才确定的，并非固定不变。

2.4 本章小结

本章系统介绍了分布式训练中的集体通信原语，从基础概念到核心操作，再到通用原语和记忆技巧，形成了完整的知识链条：

1. **集体通信的本质**：用高层次的通信模板替代繁琐的点对点编程，由底层库（如 NCCL）自动优化路由和同步。
2. **基础四原语**：Broadcast（复制）、Scatter（切分分发）、Gather（拼接收集）、Reduce（规约汇总）。它们虽非训练核心循环的常客，却是理解高级操作的必要阶梯。
3. **训练三大核心**：

⁰截图来源：讲座幻灯片，时间戳约 00:19:45。

- **All-Gather**: 每个进程收集全部数据，用于参数分片后的前向传播。
- **Reduce-Scatter**: 逐维度规约后分散结果，用于梯度汇总且避免单点显存爆炸。
- **All-Reduce**: 等价于 Reduce-Scatter + All-Gather，是数据并行中梯度同步的“主力军”。

4. **All-to-All**: 最通用的集体通信模式，在 MoE 模型的动态路由中形成矩阵转置式的通信模式。

5. **关键等价关系**: $\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather}$ 。在高级优化（ZeRO/FSDP）中，显式拆分这一等式可以在中间步骤插入自定义逻辑，获得更大的灵活性。

掌握这些原语后，我们便具备了理解后续硬件互联拓扑、NCCL 通信库以及 PyTorch 分布式编程实践的理论基础。下一章将聚焦于 GPU 集群的物理互联架构——从 PCIe 到 NVLink、NVSwitch，再到 InfiniBand 和 RDMA——揭示集体操作在硬件层面是如何被高效实现的。

3 硬件互联架构与 NCCL

前两节分别讨论了单 GPU 内部的存储层次与多 GPU 之间的集体通信原语，但尚未回答一个根本问题：不同 GPU 之间究竟通过什么物理介质交换数据？本节将沿着“节点内—节点间—集群间”的层次，系统梳理从 PCIe 到 NVLink、从 InfiniBand 到以太网的拓扑演进，并引入 RDMA 技术与 NCCL 通信库，揭示大规模 GPU 集群的硬件底座如何支撑上一节所述的集体操作。

3.1 从 PCIe 到 NVLink/InfiniBand 的拓扑演进

3.1.1 经典 PCIe 拓扑：单节点时代的局限

在 GPU 大规模集群普及之前，一台服务器的内部结构可以用一张非常经典的示意图概括(图 15)。服务器搭载多颗 CPU，通过一条 PCIe 总线连接鼠标、键盘等外设，同时挂载若干 GPU 和内存模块。不同服务器之间则通过以太网相连。

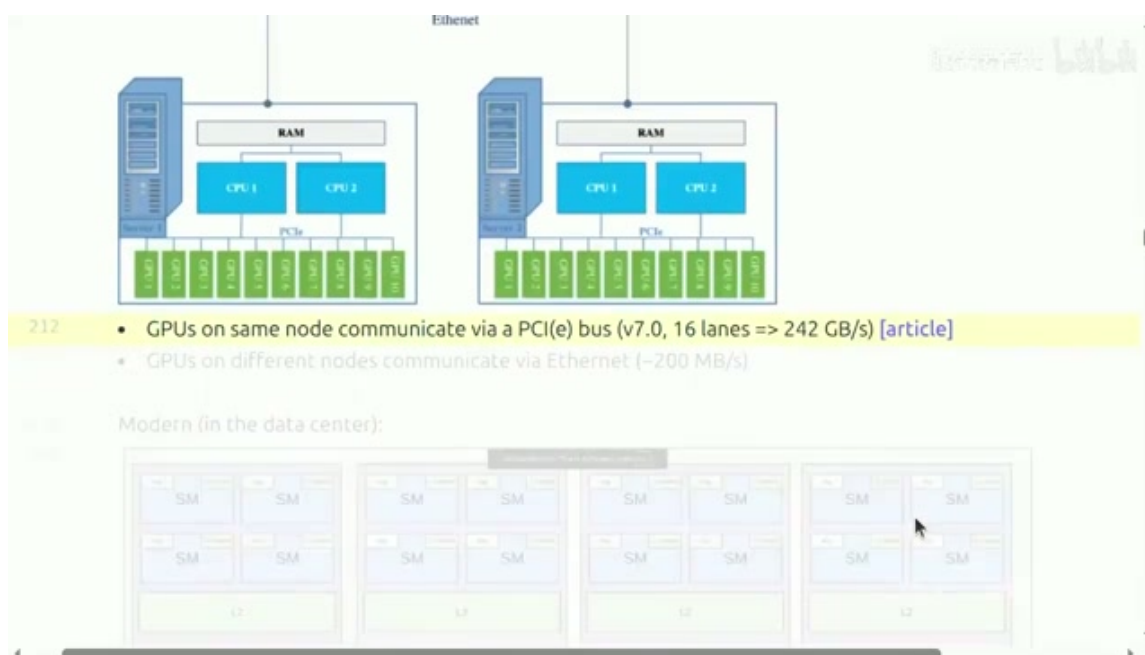


图 15: 经典服务器 PCIe 拓扑：CPU 通过 PCIe 总线连接同节点 GPU，跨节点通信依赖以太网

这种架构的核心特征是：

- **同节点通信**：GPU 之间共享 PCIe 总线（PCIe v7.0, 16 lanes $\approx$ 242 GB/s）。
- **跨节点通信**：GPU 必须将数据经 PCIe 复制到 CPU 内存，再由 CPU 通过以太网发送，速度骤降至 1–200 MB/s 量级。

从编程视角看，这相当于买了一块游戏 GPU 连到朋友的电脑上说“我要训练一个大模型”——在认真做大规模训练的场景下，这种拓扑的带宽瓶颈是不可接受的。

3.1.2 现代数据中心拓扑：NVLink + NVSwitch + InfiniBand

图 16 展示了现代 GPU 集群的典型拓扑。每个节点通常配置 8 块 GPU，这些 GPU 不再仅依赖 PCIe，而是通过 NVIDIA 专有的 NVLink 高速互联通道连接到一个 NVSwitch 交换芯片上。



图 16: 现代 GPU 集群拓扑: 单节点内 GPU 通过 NVLink 连接 NVSwitch, 跨节点通过 InfiniBand/以太网互联

NVLink 与 NVSwitch 的关键参数

- **NVLink 5.0** (以 B200 为例): 总带宽约为 **1.8 TB/s**。
- **HBM (高带宽内存)**: 带宽约为 **8 TB/s**。
- NVLink 速度约为 HBM 的 1/4, 虽远快于 PCIe 和以太网, 但仍显著慢于片上存储层次 (共享内存、L1/L2 缓存)。

从编程角度看，NVSwitch 的存在使得同一 NVLink 域内的所有 GPU 在逻辑上呈“全互联”状态：任意两块 GPU 之间的通信由硬件自动路由到交换机并转发，开发者无需关心物理走线细节。

⁰ 视频画面时间区间：00:22:09–00:22:46。

⁰ 视频画面时间区间：00:23:20–00:23:35。

3.1.3 带宽层级的逐级下降

当集群规模继续扩大，单一 NVSwitch 无法覆盖所有 GPU 时，拓扑会引入更多层级的网络互联，每一层的带宽都会显著下降（图 17）：

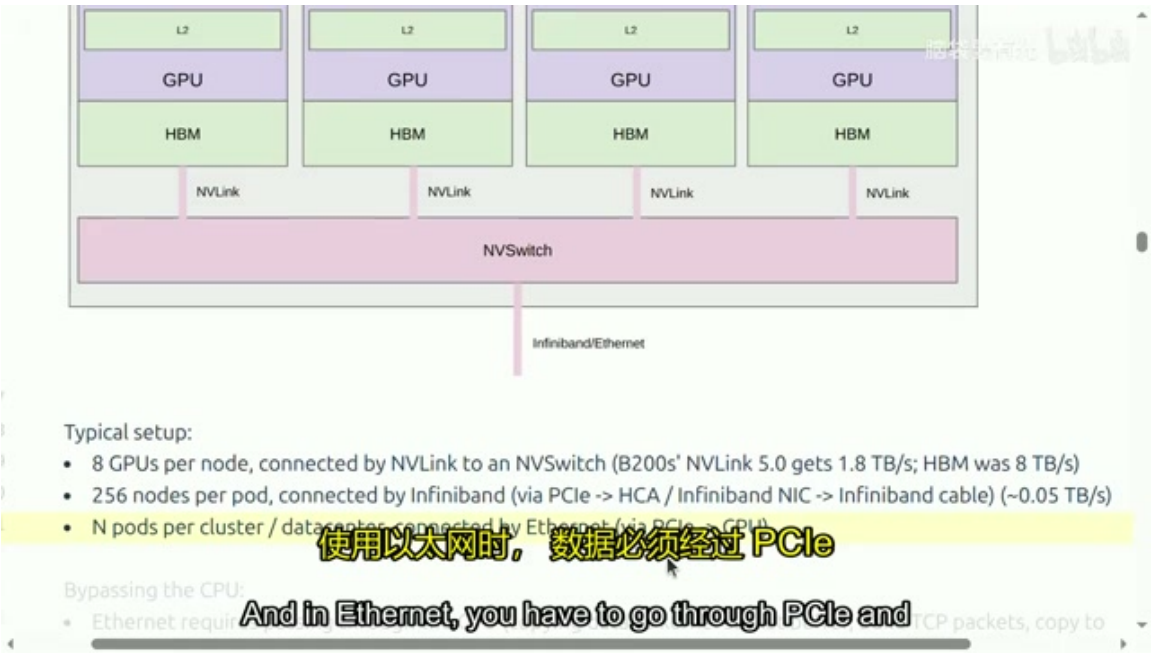


图 17: 典型大规模集群配置：带宽从 NVLink 域到以太网逐级下降

表 1: GPU 集群各层级互联带宽对比

层级	互联技术	典型带宽	数据路径
片上存储	HBM	~8 TB/s	GPU 内部
单节点内	NVLink 5.0	~1.8 TB/s	GPU → NVSwitch → GPU
跨节点 (POD 内)	InfiniBand	~0.05 TB/s	GPU → PCIe → HCA/NIC → IB 线缆
跨 POD/集群	以太网	更慢	GPU → PCIe → CPU → 以太网

POD 与集群的层级概念

- **节点 (Node)**：一台物理服务器，通常含 8 块 GPU。
- **POD**：集群的一个逻辑分组，例如 256 个节点构成一个 POD。
- **集群 (Cluster) / 数据中心**：由多个 POD 通过以太网互联构成，可扩展至数万个 GPU。

节点数越多，传输路径越长，速度越慢——这与单 GPU 内部的存储层次 (L1 → L2 → HBM) 遵循同样的“距离越远、速度越慢”规律。

⁰ 视频画面时间区间：00:25:08–00:25:58。

3.2 RDMA 与大规模集群扩展

3.2.1 传统以太网的数据路径与 CPU 瓶颈

在经典以太网通信中，GPU 无法直接发送数据到另一台机器的 GPU。数据必须经历以下冗长路径：

1. GPU 将数据复制到 CPU 主存；
2. CPU 内核构建网络数据包（写入内核套接字缓冲区）；
3. 数据经网络接口卡（NIC）发送到对端；
4. 对端 CPU 接收、解包，再复制到目标 GPU。

这里的“内核”指的是 CPU 操作系统内核，而非 GPU 计算内核。CPU 的深度介入造成了巨大的延迟开销，使得传统以太网成为大规模训练中的显著瓶颈。

3.2.2 RDMA：绕过 CPU 的直接内存访问

RDMA (Remote Direct Memory Access, 远程直接内存访问)

RDMA 允许一块 GPU 直接读写另一块 GPU 的内存，**完全不需要 CPU 参与**。从操作层面看，RDMA 描述的是一种通信行为；从硬件层面看，它可以通过多种物理介质实现：

- **NVLink/NVSwitch 域内**：天然支持 RDMA，GPU 之间直接互通。
- **InfiniBand**：支持 RDMA，GPU 可通过 IB 网卡直接收发数据。
- **标准以太网**：**不支持** RDMA，数据必须经过 CPU。

图 18 对比了两种数据路径的本质差异：

3.2.3 RoCE：以太网上的 RDMA 替代方案

InfiniBand 虽然性能优异，但价格昂贵。为了在以成本为导向的场景中实现类似 RDMA 的效果，业界发展出了 **RoCE (RDMA over Converged Ethernet, 基于融合以太网的 RDMA)**：

- RoCE 让标准以太网直接绕过 CPU 进行数据交换；
- 性能接近 InfiniBand，但成本更低；
- Meta 等公司已发表相关论文，探索在融合以太网上训练大模型的可行性。

RDMA 实现方式的辨析

NVLink、NVSwitch、InfiniBand 是**硬件层面**的零部件和线缆标准；RDMA 是**操作层面**的通信行为描述。二者属于不同抽象层次，不应混淆。例如，“RDMA over Converged Ethernet”就是以太网上实现 RDMA 行为的另一种方式。

⁰ 视频画面时间区间：00:26:15–00:27:42。

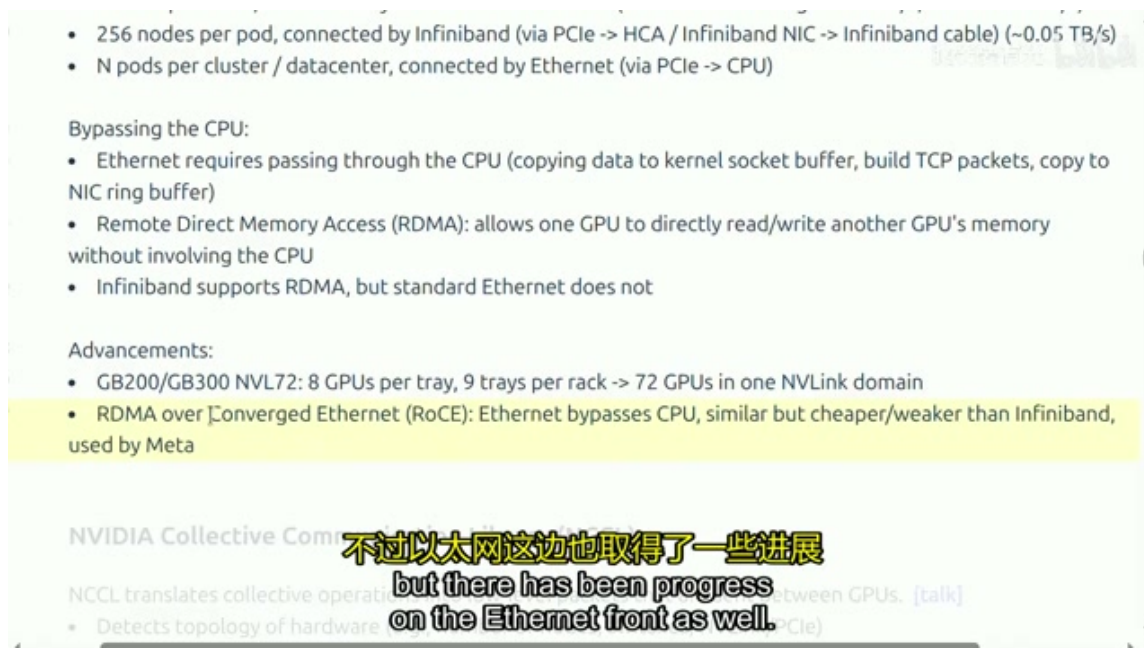


图 18: RDMA 与传统以太网的数据路径对比：RDMA 绕过 CPU，标准以太网必须经过 CPU 内核套接字缓冲区

3.3 NVL72 与 NCCL 通信库

3.3.1 NVL72：打破“八卡节点”的边界

传统认知中，一个节点通常只有 8 块 GPU，超出这个范围后通信速度会大幅下降。NVIDIA 针对 B200/B300 系列推出的 **NVL72** 方案彻底改变了这一格局：

NVL72 的硬件规格

- 每个**托盘 (Tray)** 配置 8 块 GPU；
- 一个**机架 (Rack)** 内堆叠 9 个托盘；
- 总计 $9 \times 8 = 72$ 块 GPU；
- 所有 72 块 GPU 通过 NVSwitch 连接到**同一个 NVLink 域内**。

图 19展示了 NVL72 所形成的超大规模高速互联域：

这意味着，只要预算充足，就可以在单一 NVLink 域内实现多达 72 块 GPU 的超高速互联，而无需立即跌入 InfiniBand 或以太网的慢速层级。

3.3.2 NCCL：硬件拓扑之上的集体通信抽象

无论硬件拓扑多么复杂，深度学习框架（如 PyTorch）都不应直接操作 NVLink 或 InfiniBand 的数据包。NVIDIA 提供了 **NCCL (NVIDIA Collective Communications Library, NVIDIA 集体通信库)** 作为中间层（图 20）。

NCCL 的核心职责包括：

⁰ 视频画面时间区间：00:28:00–00:29:55。

⁰ 视频画面时间区间：00:30:02–00:30:55。

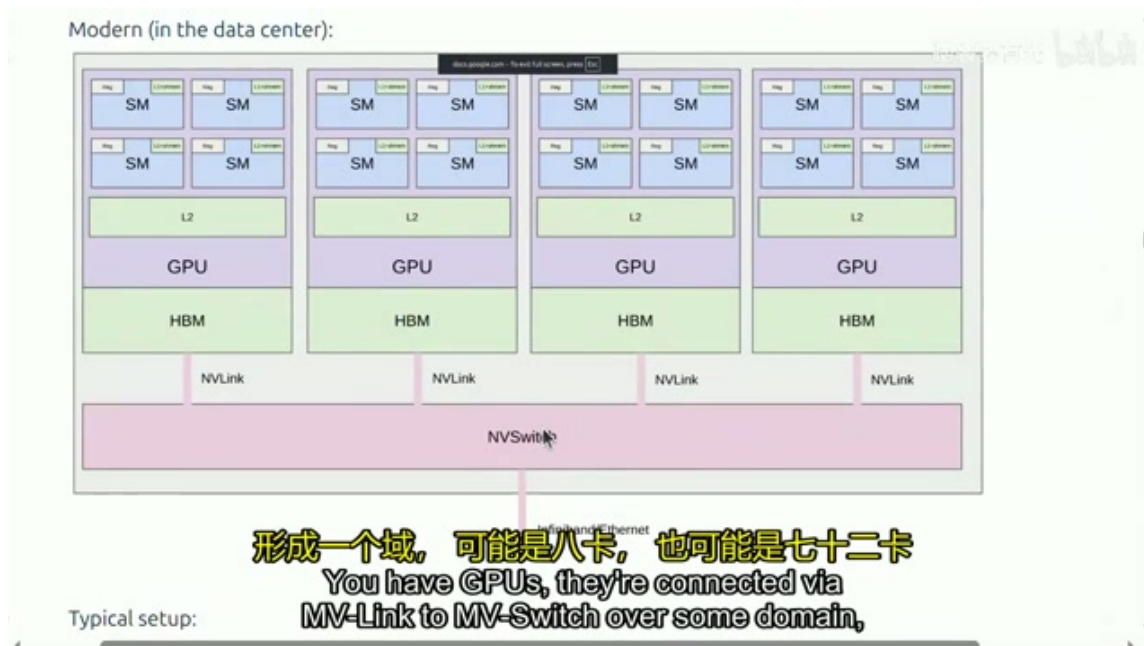


图 19: NVL72 架构：72 块 GPU 通过 NVSwitch 构成单一 NVLink 域，实现超高速全互联

- Remote Direct Memory Access (RDMA): allows one GPU to directly read/write another GPU's memory without involving the CPU
- Infiniband supports RDMA, but standard Ethernet does not

Advancements:

- GB200/GB300 NVL72: 8 GPUs per tray, 9 trays per rack -> 72 GPUs in one NVLink domain
- RDMA over Converged Ethernet (RoCE): Ethernet bypasses CPU, similar but cheaper/weaker than Infiniband, used by Meta

NVIDIA Collective Communication Library (NCCL)

NCCL translates collective operations into low-level packets that are sent between GPUs. [talk]

- Detects topology of hardware (e.g., number of nodes, switches, NVLink/PCIe)
- Optimizes the path between GPUs
- Launches GPU kernels to send/receive data

所以也存在专门的通信内核，用来和其他 GPU 进行数据交换

that actually do communication with other GPUs.

```
def torch_distributed():
    PyTorch distributed library (torch.distributed) (documentation)
    • Provides clean interface for multi-machine distributed training with torch.nn.parallel.DistributedDataParallel
```

图 20: NCCL 的功能定位：将高层集体操作转换为 GPU 之间的底层数据包

1. **操作翻译**: 将 All-Reduce、Reduce-Scatter、All-Gather、Broadcast 等高层集体操作转换为 GPU 之间实际传输的底层数据包。
2. **拓扑感知**: 自动检测硬件拓扑 (节点数、交换机数量、NVLink/PCIe 路径), 规划最优通信路径。
3. **内核启动**: 启动 GPU 上的**通信内核 (communication kernels)** 来实际发送和接收数据——记住, GPU 上运行的所有代码都是内核, 通信也不例外。

NCCL 与 PyTorch 的关系

在 PyTorch 中, `torch.distributed` 后端默认使用 NCCL 进行 GPU 间的集体通信。当用户调用:

```
dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
```

NCCL 会在幕后完成拓扑分析、路径规划和内核启动, 开发者无需关心底层是 NVLink、NVSwitch 还是 InfiniBand。

3.3.3 非对称 GPU 配置的通信困境

讲座中有一个学生提问极具启发性: “如果有九块 GPU 会怎样?” 讲师的回答揭示了拓扑对并行策略的深刻影响:

- 如果每个节点 8 卡, 第 9 卡位于另一节点且没有 NVLink 连接, 通信成本极高;
- 该节点不仅计算能力有限, 而且与其他节点的通信会成为瓶颈;
- 如果所有节点都通过 NVSwitch 互联, 则情况合理得多。

这一讨论提醒我们: **并行策略的设计必须与硬件拓扑相匹配**。强行将模型拆分到拓扑不对称的 GPU 上, 会导致某些 GPU 成为通信瓶颈, 抵消并行带来的加速收益。

3.4 本章小结

本章从硬件互联的角度, 为前两节讨论的集体通信原语提供了物理层面的解释:

1. **拓扑演进**: 从经典 PCIe+ 以太网到现代 NVLink+NVSwitch+InfiniBand, GPU 集群的互联带宽实现了数量级的跃升。单节点内 NVLink (~ 1.8 TB/s) 远快于 PCIe, 但仍是 HBM (~ 8 TB/s) 的约 1/4。
2. **带宽层级**: 节点内 (NVLink) > POD 内 (InfiniBand, ~ 0.05 TB/s) > 跨 POD (以太网, 更慢)。层级越多、距离越远, 速度越慢——与单 GPU 存储层次遵循同一规律。
3. **RDMA 技术**: 允许 GPU 直接读写远端 GPU 内存, 绕过 CPU。NVLink/NVSwitch 和 InfiniBand 均支持 RDMA; 标准以太网不支持, 但 RoCE 提供了低成本的替代路径。
4. **NVL72**: 通过 9 托盘 $\times$ 8 GPU 的配置, 将 72 块 GPU 纳入单一 NVLink 域, 打破了传统“八卡节点”的带宽边界。
5. **NCCL**: 作为硬件拓扑与高层集体操作之间的桥梁, 自动检测拓扑、规划路径、启动 GPU 通信内核, 是 PyTorch 分布式训练的底层基石。

核心 takeaway: 在多 GPU 训练中，通信开销往往成为比计算更硬的瓶颈。理解硬件拓扑的带宽层级，并选择与拓扑匹配的并行策略（如将通信密集的 All-Reduce 限制在 NVLink 域内），是实现高效分布式训练的前提。下一节将进入 PyTorch 分布式编程实践，展示如何在代码层面调用这些硬件能力。

4 PyTorch 分布式编程实践

前三节分别从编程模型（集体通信原语）和硬件底座（NVLink、NVSwitch、InfiniBand、NCCL）两个角度建立了分布式训练的理论基础。本节将两者桥接起来：我们进入 PyTorch 代码层面，展示如何通过 `torch.distributed` 库调用底层 NCCL/Gloo 能力，实现 All-Reduce、Reduce-Scatter 与 All-Gather 等核心集体操作，并学习如何测量实际通信带宽，为后续并行策略的选型提供量化依据。

4.1 torch.distributed 基础与进程管理

4.1.1 动机：从理论原语到可执行代码

集体通信原语（All-Reduce、All-Gather 等）描述的是“做什么”，但要在真实 GPU 集群上运行，还需要解决以下工程问题：

1. 如何在多台机器上启动和管理多个进程？
2. 如何让这些进程彼此发现对方并建立通信通道？
3. 如何根据硬件类型（GPU vs. CPU）选择正确的通信后端？
4. 如何在异步执行的多进程环境中保证同步与数据一致性？

PyTorch 通过 `torch.distributed` 模块为上述问题提供了简洁而统一的解决方案。该模块封装了底层通信库（NCCL、Gloo），向用户暴露高层次的集体通信 API，使开发者无需关心数据包如何在 NVLink 或 InfiniBand 上传输。

4.1.2 核心概念：Rank、World Size 与后端选择

在 `torch.distributed` 的语境中，我们沿用集体通信的标准术语：

- **Rank**：当前进程的编号，从 0 开始，范围是 0 到 $N - 1$ 。
- **World Size (N)**：参与分布式训练的进程总数，通常等于 GPU 总数。

`torch.distributed` 支持多种后端，以适配不同的硬件环境：

- **NCCL 后端**(NVIDIA Collective Communications Library): 专为 GPU 设计, 利用 NVLink/NVSwitch/InfiniBand 实现高速集体通信。这是 GPU 分布式训练的**首选后端**。
- **Gloo 后端**：由 Facebook 开发的通用集体通信库，支持 CPU 和 GPU，但 GPU 性能不如 NCCL。当环境没有 NVIDIA GPU 时（如在笔记本电脑上演示），Gloo 是可靠的回退方案。

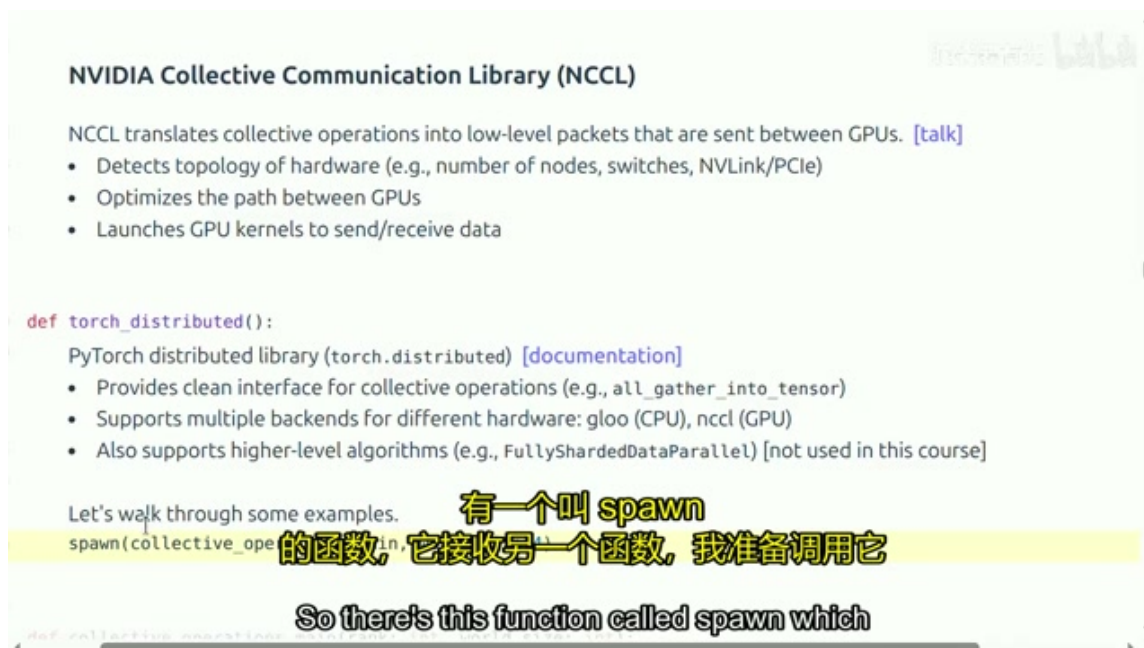


图 21: torch.distributed 的功能定位：为集体通信提供简洁接口，支持 NCCL（GPU）和 Gloo（CPU）后端。

此外，PyTorch 还内置了更高层次的封装，如 FullyShardedDataParallel (FSDP)，但本课程选择从零开始手写分布式代码，以便深入理解底层机制。

4.1.3 进程启动：spawn 函数

在多进程分布式程序中，通常为每块 GPU 创建一个独立的进程。PyTorch 提供了 torch.multiprocessing 函数来完成这一任务：

```

1 import torch.multiprocessing as mp
2
3 def collective_operations_main(rank: int, world_size: int):
4     """该函数在每个进程上异步执行(rank=0, ..., world_size-1)"""
5     setup(rank, world_size) # 初始化分布式环境
6     # ... 执行集体通信操作 ...
7     cleanup() # 清理分布式资源
8
9 if __name__ == "__main__":
10     world_size = 4
11     mp.spawn(collective_operations_main, args=(world_size,), nprocs=world_size)

```

Listing 1: 使用 spawn 启动多进程分布式程序

spawn 接收一个目标函数（此处为 collective_operations_main），并将其复制 world_size 份，分别在独立的进程中运行。每个进程会被分配一个唯一的 rank 参数，范围从 0 到 $N - 1$ 。这些进程是异步执行的——我们无法预测哪个进程会先完成，它们的完成顺序由操作系统调度器和硬件状态共同决定。

⁰ 视频截图来源：00:36:50。

spawn 与单进程演示模式

在讲座的逐步讲解中，讲师使用了一个特殊的单进程演示模式（通过 `python -m edtrace.execute -m lecture_07` 运行），以便在无法使用多进程的环境中展示代码逻辑。实际训练时应使用 `torch.multiprocessing.spawn` 或 `torchrun` 启动真正的多进程。

4.1.4 初始化: `init_process_group`

每个进程启动后，第一步是调用 `init_process_group` 来初始化分布式环境：

```
1 import os
2 import torch.distributed as dist
3
4 def setup(rank: int, world_size: int):
5     """初始化分布式环境（在每个进程开始时调用）"""
6     # 配置主节点地址和端口，用于进程间协调
7     os.environ["MASTER_ADDR"] = "localhost"
8     os.environ["MASTER_PORT"] = "15623"
9
10    if torch.cuda.is_available():
11        # GPU 环境：使用 NCCL 后端
12        dist.init_process_group("nccl", rank=rank, world_size=world_size)
13    else:
14        # CPU 环境：使用 Gloo 后端
15        dist.init_process_group("gloo", rank=rank, world_size=world_size)
```

Listing 2: 初始化分布式进程组

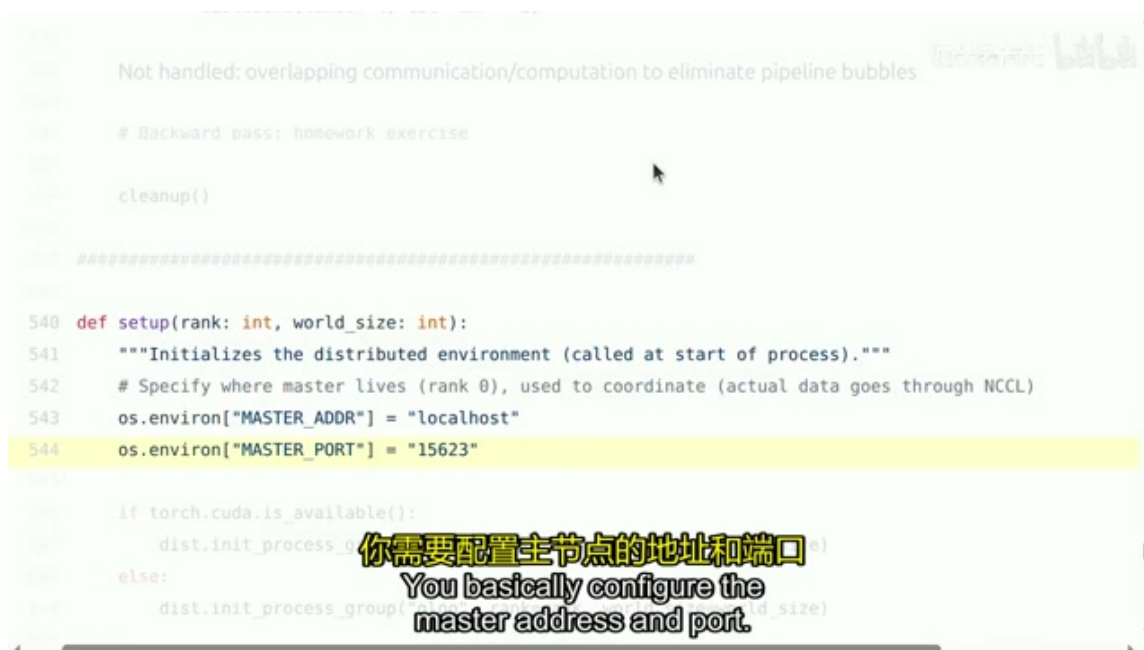


图 22: 分布式环境初始化代码：配置主节点地址和端口，根据硬件选择 NCCL 或 Gloo 后端。

⁰ 视频截图来源：00:38:43。

需要特别注意的是, MASTER_ADDR 和 MASTER_PORT 仅用于**进程间的元数据协调与同步** (如建立连接、交换地址信息), **实际的数据传输并不经过这个 TCP 通道**。真正的数据通信由 NCCL 后端通过 NVLink 或 InfiniBand 完成, 否则速度将极其缓慢。

4.1.5 同步屏障: barrier 的作用与开销

由于各进程异步执行, 我们无法控制哪个进程先到达某一行代码。如果希望确保某些操作在其他操作之前完成, 就需要插入**同步屏障** (barrier):

```
1 dist.barrier() # 等待所有进程都到达这一点
```

Listing 3: 同步屏障的使用

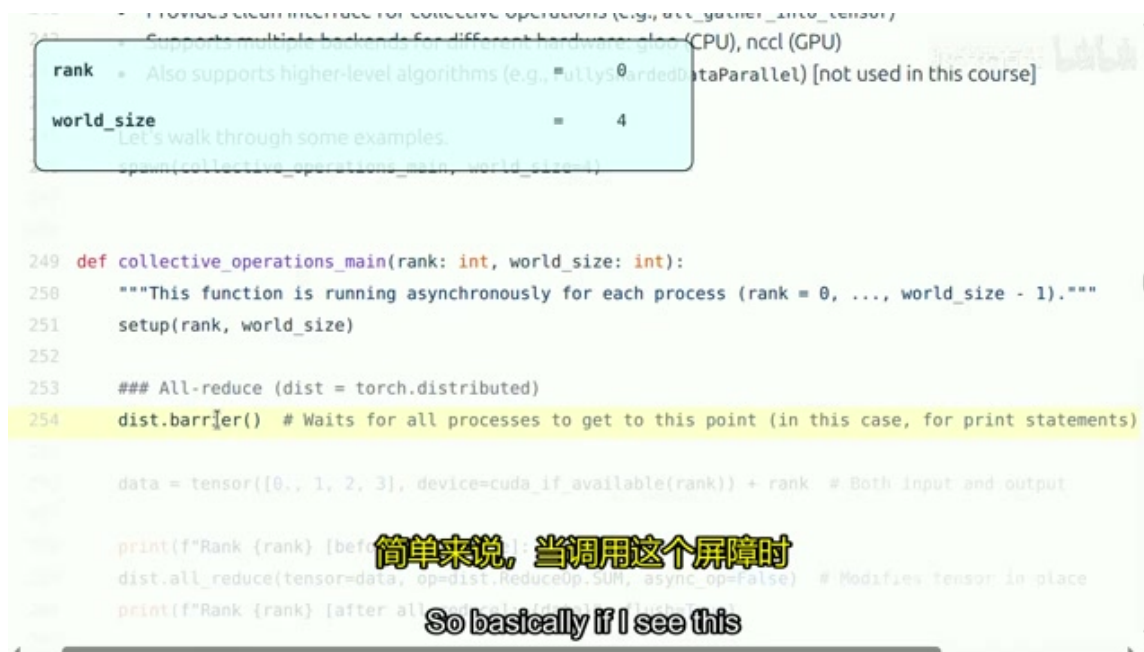


图 23: 代码中的 `dist.barrier()` 调用: 确保所有进程在继续执行前到达同步点。

`barrier` 的语义很简单: 当任一进程调用 `barrier` 时, 它会阻塞并等待, 直到**所有**进程都调用了 `barrier`, 然后所有进程同时继续执行。这在以下场景中至关重要:

- 在打印调试信息前, 确保所有进程都已准备好相关数据, 避免输出交错混乱。
- 在计时基准测试前, 确保所有前置操作 (如预热) 已完成。
- 在参数更新前, 确保所有进程的梯度同步已完成。

然而, 过多的 `barrier` 也会带来**不必要的等待开销**——最快的进程必须等待最慢的进程。因此, 在实际训练中应谨慎使用屏障, 尽量让计算与通信重叠, 而非频繁同步。

⁰ 视频截图来源: 00:39:16。

4.2 All-Reduce、Reduce-Scatter 与 All-Gather 代码实现

4.2.1 All-Reduce：就地求和并广播

动机：在数据并行训练中，每个 GPU 计算出的梯度需要汇总求和，然后广播给所有 GPU，以确保各进程的参数更新保持一致。All-Reduce 正是完成这一任务的原语。

代码实现：

```
1  ### All-Reduce (dist = torch.distributed)
2  dist.barrier() # 等待所有进程到达此点（确保打印顺序可控）
3
4  # 每个rank创建不同的输入数据：rank 0 得到 [0,1,2,3]，rank 1 得到 [1,2,3,4]，...
5  data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank
6
7  print(f"Rank_{rank}_[before_all-reduce]:_{data}", flush=True)
8
9  # 执行All-Reduce：就地求和，async_op=False 表示同步执行
10 dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
11
12 print(f"Rank_{rank}_[after_all-reduce]:_{data}", flush=True)
```

Listing 4: All-Reduce 操作示例

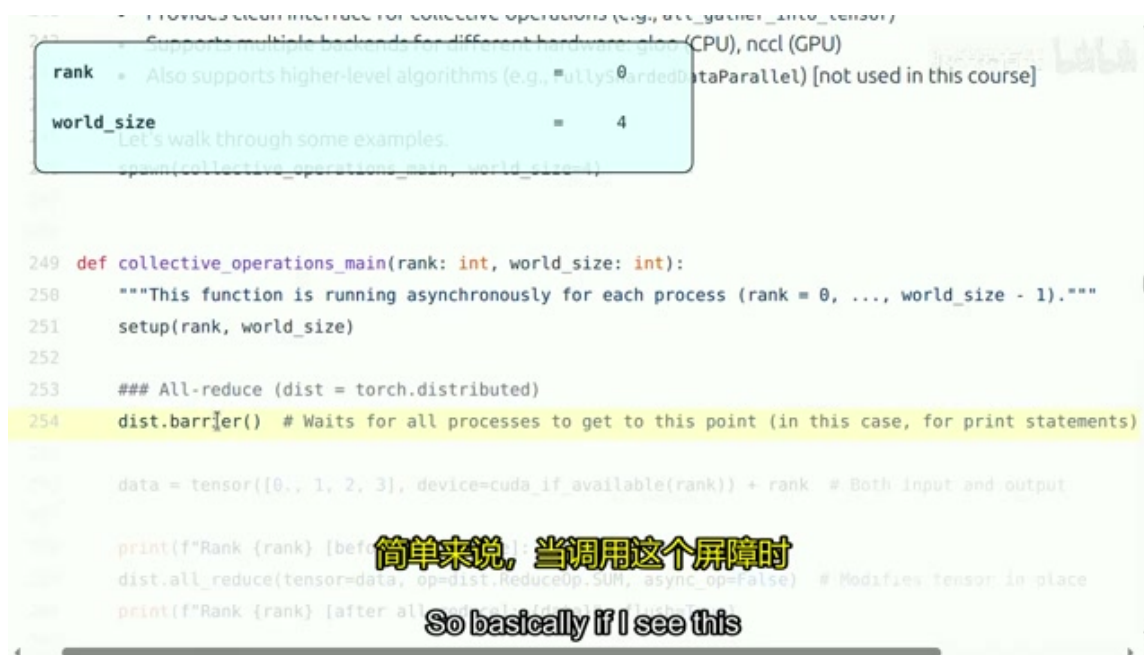


图 24: All-Reduce 代码示例：每个 rank 创建不同的输入张量，执行求和规约后所有 rank 获得相同结果。

⁰视频截图来源：00:40:26。

执行结果分析：假设 `world_size = 4`，All-Reduce 前的输入为：

Rank 0 : [0, 1, 2, 3]

Rank 1 : [1, 2, 3, 4]

Rank 2 : [2, 3, 4, 5]

Rank 3 : [3, 4, 5, 6]

对每列求和：第 0 列 $0 + 1 + 2 + 3 = 6$ ，第 1 列 $1 + 2 + 3 + 4 = 10$ ，第 2 列 $2 + 3 + 4 + 5 = 14$ ，第 3 列 $3 + 4 + 5 + 6 = 18$ 。All-Reduce 后，**每个 Rank** 都持有完整的结果：

Rank 0-3 : [6, 10, 14, 18]



```
Rank 0 [after reduce-scatter]: input = tensor([0., 1., 2., 3.], device='cuda:0'), output = tensor([0.], device='cuda:0')
Rank 1 [after reduce-scatter]: input = tensor([1., 2., 3., 4.], device='cuda:1'), output = tensor([10.], device='cuda:1')
Rank 3 [after reduce-scatter]: input = tensor([3., 4., 5., 6.], device='cuda:3'), output = tensor([18.], device='cuda:3')
Rank 2 [after reduce-scatter]: input = tensor([2., 3., 4., 5.], device='cuda:2'), output = tensor([14.], device='cuda:2')
Rank 0 [before all-gather]: input = tensor([6.], device='cuda:0'), output = tensor([ 6., 10., 14., 18.], device='cuda:0')
Rank 2 [before all-gather]: input = tensor([14.], device='cuda:2'), output = tensor([ 6., 10., 14., 18.], device='cuda:2')
Rank 1 [before all-gather]: input = tensor([10.], device='cuda:1'), output = tensor([ 6., 10., 14., 18.], device='cuda:1')
Rank 3 [before all-gather]: input = tensor([18.], device='cuda:3'), output = tensor([ 6., 10., 14., 18.], device='cuda:3')
Rank 0 [after all-gather]: input = tensor([6.], device='cuda:0'), output = tensor([ 6., 10., 14., 18.], device='cuda:0')
Rank 2 [after all-gather]: input = tensor([14.], device='cuda:2'), output = tensor([ 6., 10., 14., 18.], device='cuda:2')
Rank 1 [after all-gather]: input = tensor([10.], device='cuda:1'), output = tensor([ 6., 10., 14., 18.], device='cuda:1')
Rank 3 [after all-gather]: input = tensor([18.], device='cuda:3'), output = tensor([ 6., 10., 14., 18.], device='cuda:3')
[all_reduce] Rank 0: all_reduce(world_size=4, num_elements=104857600) took 1.60ms
[all_reduce] Rank 2: all_reduce(world_size=4, num_elements=104857600) took 1.38ms
[all_reduce] Rank 1: all_reduce(world_size=4, num_elements=104857600) took 1.50ms
[all_reduce] Rank 3: all_reduce(world_size=4, num_elements=104857600) took 1.38ms
[all_reduce] Rank 1: all_reduce measured bandwidth = 390 GB/s
[all_reduce] Rank 2: all_reduce measured bandwidth = 426 GB/s
[all_reduce] Rank 0: all_reduce measured bandwidth = 366 GB/s
[all_reduce] Rank 3: all_reduce measured bandwidth = 425 GB/s
[reduce_scatter] Rank 0: reduce_scatter(world_size=4, num_elements=104857600) took 2.61ms
[reduce_scatter] Rank 1: reduce_scatter(world_size=4, num_elements=104857600) took 2.47ms
[reduce_scatter] Rank 2: reduce_scatter(world_size=4, num_elements=104857600) took 2.39ms
[reduce_scatter] Rank 3: reduce_scatter(world_size=4, num_elements=104857600) took 2.39ms
[reduce_scatter] Rank 1: reduce_scatter measured bandwidth = 475 GB/s
[reduce_scatter] Rank 0: reduce_scatter measured bandwidth = 490 GB/s
[reduce_scatter] Rank 2: reduce_scatter measured bandwidth = 490 GB/s
[reduce_scatter] Rank 3: reduce_scatter measured bandwidth = 490 GB/s
```

图 25: 终端输出：展示了 All-Reduce、Reduce-Scatter 和 All-Gather 在各 rank 上的输入输出结果，以及基准测试的耗时与带宽测量值。

注意，打印语句的输出顺序由硬件异步执行随机决定，但数据内容始终正确。

`dist.all_reduce` 的关键参数说明：

- `tensor`: 输入输出张量（就地修改）。
- `op`: 规约操作类型，如 `dist.ReduceOp.SUM`（求和）、`AVG`（平均，部分版本支持）。
- `async_op`: 是否异步执行。`False` 表示阻塞直到操作完成；`True` 表示立即返回，允许计算与通信重叠。

4.2.2 Reduce-Scatter: 输入输出分离的规约散播

动机: 与 All-Reduce 不同，Reduce-Scatter 将输入和输出分离——输入张量保持不变，规约结果写入独立的输出张量，且每个 rank 只获得规约结果的一个分片。这在显存优化场景（如 ZeRO/FSDP）中非常有用。

⁰ 视频截图来源：00:43:37。

代码实现：

```
1 dist.barrier()
2
3 # 输入：每个rank持有一个长度为 world_size 的向量
4 input = torch.arange(world_size, device=cuda_if_available(rank)) + rank
5 # 输出：预先分配空间
6 output = torch.empty(1, device=cuda_if_available(rank))
7
8 print(f"Rank_{rank} [before_reduce-scatter]: input={input}, output={output}",
9       flush=True)
10
11 # 执行Reduce-Scatter：输入保持不变，输出写入规约结果的分片
12 dist.reduce_scatter_tensor(output=output, input=input,
13                             op=dist.ReduceOp.SUM, async_op=False)
14
15 print(f"Rank_{rank} [after_reduce-scatter]: input={input}, output={output}",
16       flush=True)
```

Listing 5: Reduce-Scatter 操作示例

执行结果分析：输入与 All-Reduce 相同。Reduce-Scatter 对每列求和后，将结果分散到不同 rank：

Rank 0 : output = [6] （第 0 列之和）
Rank 1 : output = [10] （第 1 列之和）
Rank 2 : output = [14] （第 2 列之和）
Rank 3 : output = [18] （第 3 列之和）

输入张量 `input` 保持不变，输出张量 `output` 仅包含一个标量值——这是 Reduce-Scatter 与 All-Reduce 的核心区别。

4.2.3 All-Gather：收集分散结果

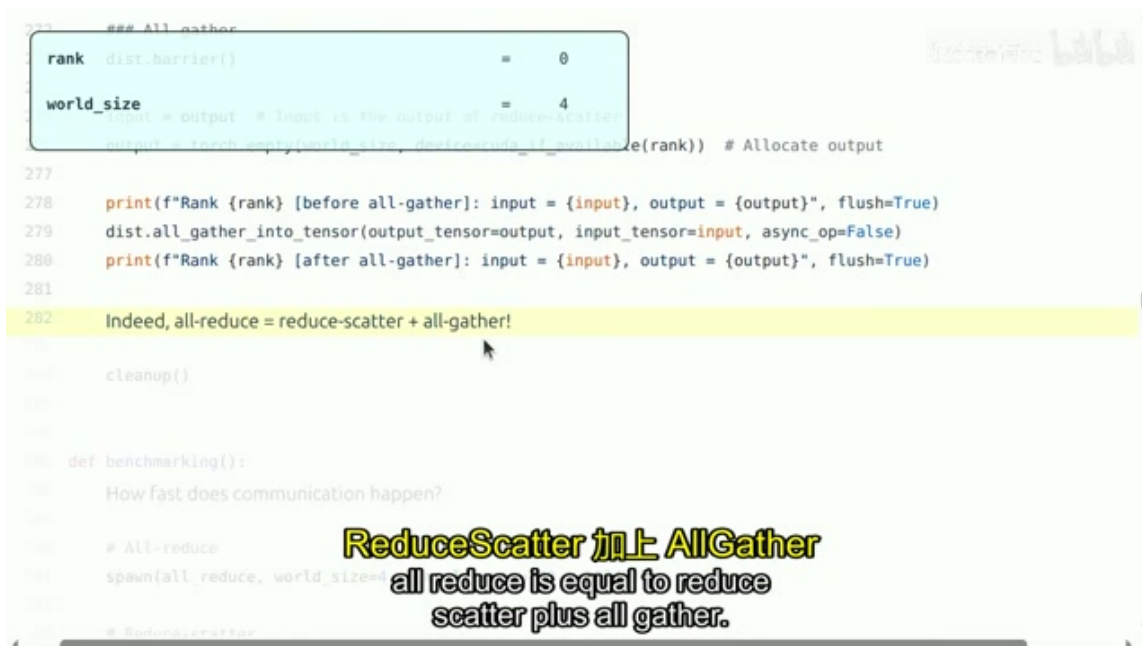
动机：在 Reduce-Scatter 之后，每个 rank 只持有规约结果的一个分片。如果后续计算需要完整的规约结果，就需要通过 All-Gather 将各 rank 上的分片收集起来。

代码实现：

```
1 # 使用 Reduce-Scatter 的输出作为 All-Gather 的输入
2 input = output # 每个rank持有一个标量：rank 0 有 [6], rank 1 有 [10], ...
3 output = torch.empty(world_size, device=cuda_if_available(rank)) # 分配输出空间
4
5 print(f"Rank_{rank} [before_all-gather]: input={input}, output={output}", flush
6       =True)
7
8 dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=
9                             False)
10
11 print(f"Rank_{rank} [after_all-gather]: input={input}, output={output}", flush=
12       True)
```

Listing 6: All-Gather 操作示例

执行结果：All-Gather 后，每个 rank 的 output 都包含完整的拼接结果 [6, 10, 14, 18]。



```

277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

图 26: 代码中明确标注的等价关系：all-reduce = reduce-scatter + all-gather。

4.2.4 核心等价关系的代码验证

讲座中明确给出了以下等价关系：

$$\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather} \quad (2)$$

这一等价关系不仅在理论层面成立，上述代码也直接验证了它：先执行 Reduce-Scatter 将规约结果分散到各 rank，再执行 All-Gather 将这些分散的分片收集并广播到所有 rank，最终效果与直接执行 All-Reduce 完全一致。

为什么需要拆分 All-Reduce？

虽然 NCCL 对 All-Reduce 有专门优化的实现，但在高级技术如 ZeRO/FSDP 中，显式地将 All-Reduce 拆分为 Reduce-Scatter 和 All-Gather 两个独立步骤，可以在中间插入额外的优化逻辑——例如梯度压缩、量化、或计算与通信的重叠调度——从而获得更大的灵活性和性能提升。

4.3 异步操作与有效带宽基准测试

4.3.1 异步集体通信：计算与通信重叠

在前面的示例中，我们将 `async_op` 设为 `False`，这意味着集体通信操作会阻塞直到完成。然而，在实际训练中，这种同步等待会浪费宝贵的 GPU 计算时间。

⁰ 视频截图来源：00:45:04。

PyTorch 支持异步集体通信：

```
1 # 启动异步All-Reduce，立即返回
2 handle = dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=True)
3
4 # 在通信进行的同时，执行其他独立的计算任务
5 # 例如：加载下一批次数据、执行不依赖当前张量的计算等
6
7 # 当需要确保通信完成时，调用等待操作
8 handle.wait()
9 # 或者使用 barrier 进行同步
10 dist.barrier()
```

Listing 7: 异步 All-Reduce

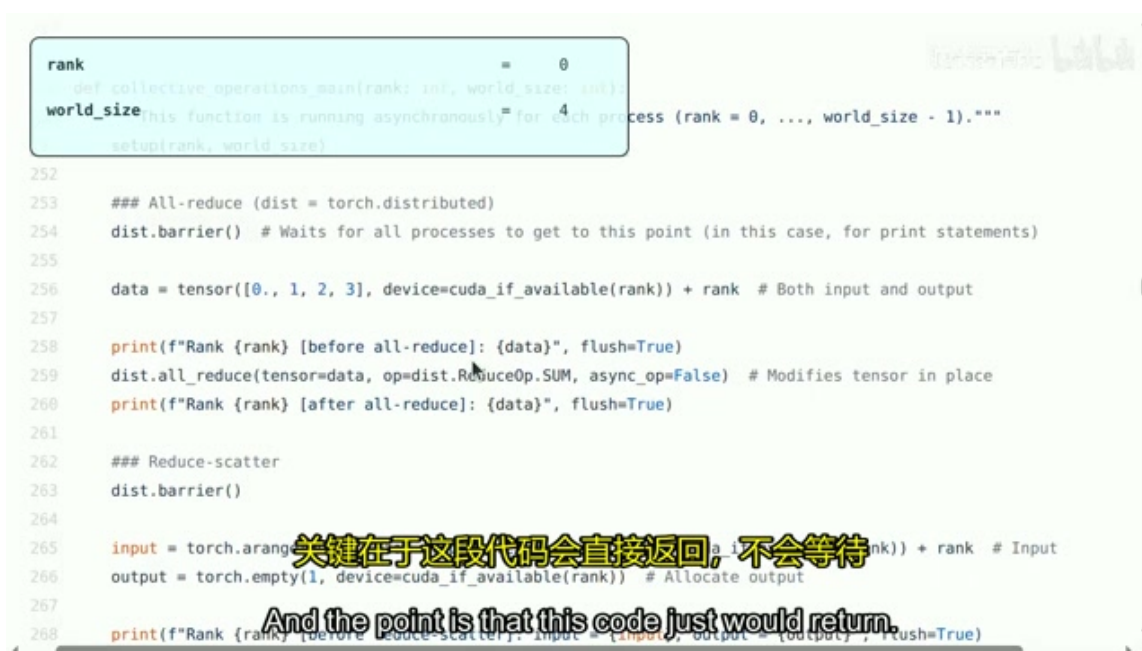


图 27: 异步操作代码：设置 `async_op=True` 后，All-Reduce 调用立即返回，允许与后续计算重叠。

核心机制：当 `async_op=True` 时，`dist.all_reduce` 会启动底层的 NCCL 通信内核（CUDA kernel），然后将控制权交还给 Python 进程。由于 NCCL 内核本身相对于 CPU 进程就是异步执行的，这意味着：

1. GPU 在后台执行通信操作；
2. CPU 进程可以同时调度其他 CUDA 计算内核；
3. 当后续计算确实需要 All-Reduce 的结果时，再调用 `handle.wait()` 或 `dist.barrier()` 进行同步。

这种**计算与通信重叠**（communication-computation overlap）是高性能分布式训练的关键技术之一。例如，在数据并行训练中，可以在 All-Reduce 梯度同步的同时，开始加载下一批次的数据或执行前向传播中不依赖同步结果的部分计算。

⁰视频截图来源：00:44:19。

异步操作中的同步陷阱

讲座中有一个学生提问极具启发性：“如果先调用 `barrier`，但此时 `CUDA` 内核还没算完，会发生什么？”讲师指出，这种情况下 `barrier` 实际上并没有真正同步——因为 `barrier` 只同步进程到达的状态，而不等待 `CUDA` 内核完成。正确的做法是先调用 `torch.cuda.synchronize()` 等待 `CUDA` 内核完成，再调用 `dist.barrier()` 等待所有进程到达同步点。顺序错误会导致计时结果不准确，甚至产生数据竞争。

4.3.2 基准测试方法：预热、计时与同步

要准确测量集体通信的性能，必须遵循严格的基准测试流程，以消除两种异步带来的干扰：

1. **CUDA 内核异步**：CUDA 操作默认异步执行，调用返回不代表内核已完成。
2. **多进程异步**：各进程独立运行，完成时间可能不同。

```
1 def all_reduce(rank: int, world_size: int, num_elements: int):
2     setup(rank, world_size)
3
4     # 创建张量
5     data = torch.randn(num_elements, device=cuda_if_available(rank))
6
7     # 预热：确保GPU已就绪
8     dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
9     torch.cuda.synchronize() # 等待CUDA内核完成
10    dist.barrier()           # 等待所有进程到达此点
11
12    # 正式计时
13    start_time = time.time()
14    dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
15    torch.cuda.synchronize() # 等待CUDA内核完成
16    dist.barrier()           # 等待所有进程到达此点
17    end_time = time.time()
18
19    duration = end_time - start_time
20    print(f"[all_reduce] Rank_{rank}: world_size={world_size}, "
21          f"num_elements={num_elements} took {render_duration(duration)}", flush=
22          True)
```

Listing 8: All-Reduce 基准测试代码

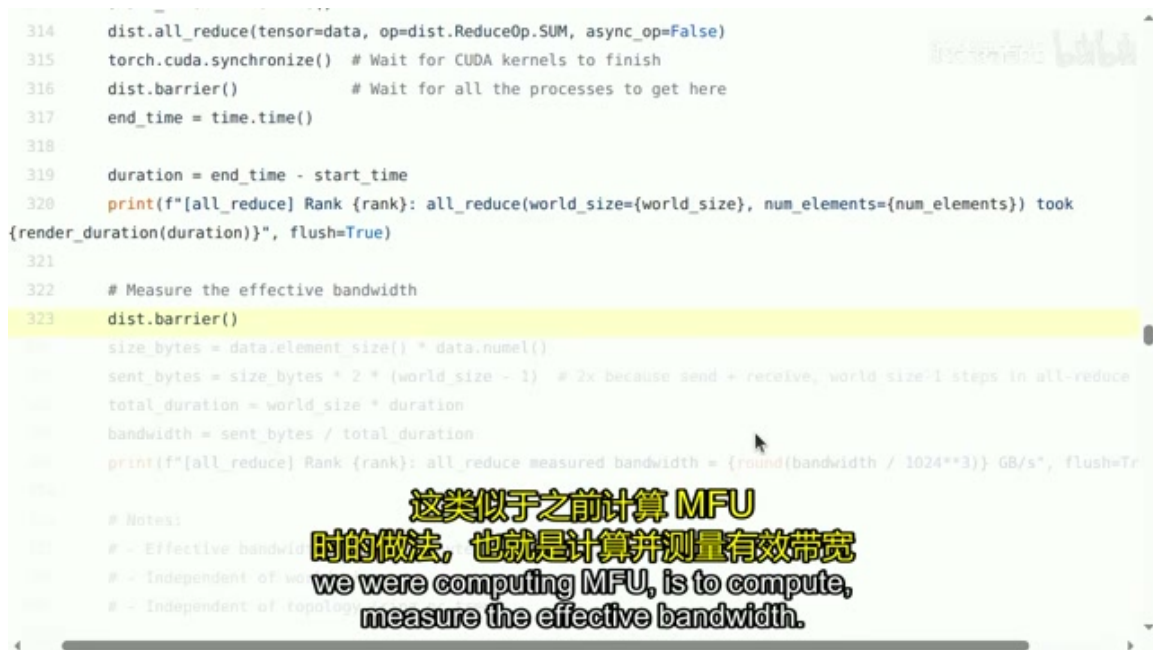


图 28: All-Reduce 基准测试代码：包含预热、cuda.synchronize 同步、barrier 同步和精确计时。

预热阶段的目的是让 GPU 进入稳定工作状态，避免首次启动内核的冷启动开销影响测量结果。**计时阶段**前后各放置一对 `torch.cuda.synchronize()` 和 `dist.barrier()`，确保计时区间内只包含 All-Reduce 操作的实际耗时。

4.3.3 有效带宽的计算

测量到耗时后，下一个问题是：这个速度到底快还是慢？为了回答这个问题，我们需要计算**有效带宽**（Effective Bandwidth）。

核心思路：计算在整个通信过程中实际传输了多少字节，然后用总字节数除以总时间，得到有效带宽。

⁰视频截图来源：00:47:14。

```

dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
torch.cuda.synchronize() # Wait for CUDA kernels to finish
dist.barrier()           # Wait for all the processes to get here
end_time = time.time()

duration = end_time - start_time
print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took
r_duration(duration))", flush=True)

# Measure the effective bandwidth
dist.barrier()
size_bytes = data.element_size() * data.numel()
sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send + receive, world_size-1 steps in all-reduce
total_duration = world_size * duration
bandwidth = sent_bytes / total_duration
print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024**3)} GB/s", flush=True)

# Notes:
# - Effective bandwidth ~ 2
# - Independent of world size
# - Independent of topology (ring)

```

**而带宽就是发送的总字节数除以总时间
and the bandwidth is the bytes
sent divided by the total duration.**

图 29: 有效带宽计算公式: $\text{sent_bytes} = \text{size_bytes} * 2 * (\text{world_size} - 1)$, 考虑了发送与接收双向数据流。

对于 All-Reduce 操作, 有效带宽的计算公式为:

$$\text{sent_bytes} = \underbrace{\text{size_bytes}}_{\text{张量总字节数}} \times \underbrace{2}_{\text{发送 + 接收}} \times \underbrace{(N - 1)}_{\text{迭代次数}} \quad (3)$$

其中:

- $\text{size_bytes} = \text{element_size} \times \text{num_elements}$, 即张量的总字节数。
- 因子 2 表示每次迭代中既需要发送数据, 又需要接收规约后的结果。
- 因子 $(N - 1)$ 表示在 All-Reduce 的朴素实现中, 需要进行 $N - 1$ 次迭代来完成所有进程的规约。

总持续时间需要考虑所有进程:

$$\text{total_duration} = N \times \text{duration} \quad (4)$$

最终带宽为:

$$\text{BW} = \frac{\text{sent_bytes}}{\text{total_duration}} = \frac{\text{size_bytes} \times 2 \times (N - 1)}{N \times \text{duration}} \quad (5)$$

当 N 较大时, $(N - 1)/N \approx 1$, 公式简化为:

$$\text{BW} \approx \frac{2 \times \text{size_bytes}}{\text{duration}} \quad (6)$$

⁰视频截图来源: 00:50:01。

有效带宽的关键性质

1. 与进程数无关：当 N 增大时， $(N - 1)/N \rightarrow 1$ ，带宽趋近于 $2 \times \text{size_bytes}/\text{duration}$ ，与 N 无关。
2. 与网络拓扑无关：无论 NCCL 选择环形拓扑（ring）还是树形拓扑（tree）来实现 All-Reduce，最终测得的有效带宽都由 NCCL 的优化策略决定，对用户透明。

在讲座的实测中，对约 1 亿个元素执行 All-Reduce，测得的有效带宽约为 **400 GB/s**。这一数值与 NVLink 的理论带宽（约 1.8 TB/s）相比仍有差距，原因是 All-Reduce 涉及多次数据搬移和规约计算，实际效率受多种因素影响。

4.3.4 Reduce-Scatter 的带宽计算

Reduce-Scatter 的带宽计算与 All-Reduce 非常相似，但有一个关键区别——**没有因子 2**：

```
1 # 输入数据的总字节数
2 data_bytes = input.element_size() * input.numel()
3
4 # 实际发送的字节数（没有2倍系数）
5 sent_bytes = data_bytes * (world_size - 1)
6
7 # 总持续时间
8 total_duration = world_size * duration
9
10 # 带宽
11 bandwidth = sent_bytes / total_duration
```

Listing 9: Reduce-Scatter 带宽计算

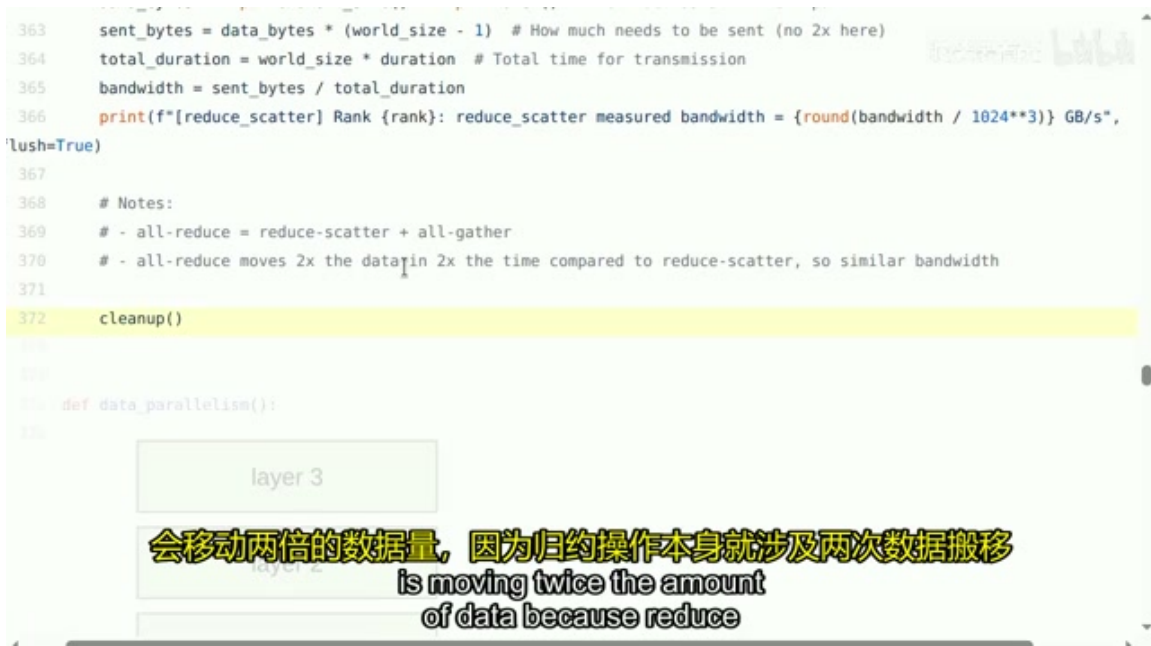


图 30: Reduce-Scatter 基准测试与带宽计算代码：注意 `sent_bytes` 没有 $2\times$ 系数。

为什么没有因子 2？ 因为 Reduce-Scatter 只涉及单向的数据搬移——将输入数据发送给其他进程进行规约，而不需要像 All-Reduce 那样再把结果广播回来。

4.3.5 All-Reduce 与 Reduce-Scatter 的带宽对比

讲座最后总结了两者的关系：

$$\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather} \quad (7)$$

这意味着：

- All-Reduce 需要移动**两倍**的数据量（Reduce-Scatter 搬一次，All-Gather 再搬一次）。
- All-Reduce 花费**两倍**的时间。
- 但由于数据量和时间同比例增加，**有效带宽保持不变**。

实测结果验证了这一结论：All-Reduce 和 Reduce-Scatter 测得的带宽都在 **400 GB/s** 左右，略有波动，但基本处于同一量级。

带宽测量的核心启示

1. 有效带宽是衡量分布式通信性能的核心指标，计算方式为 $\text{BW} = \frac{\text{sent_bytes}}{\text{total_duration}}$ 。
2. All-Reduce 的带宽公式包含 $2\times$ 系数，反映其双向数据搬移特性；Reduce-Scatter 没有该系数。
3. 带宽与进程数 N 和网络拓扑无关（由 NCCL 自动优化），这简化了性能预测和硬件选型。
4. 基准测试时必须严格同步（`cuda.synchronize + barrier`），否则测量结果会包含未完成的异步操作，导致严重偏差。

4.4 本章小结

本节从 PyTorch 代码层面深入实践了分布式集体通信，建立了从 API 调用到底层硬件执行的完整认知链条：

1. **torch.distributed 架构**：提供简洁的集体通信 API，支持 NCCL（GPU）和 Gloo（CPU）后端，将高层操作映射到底层通信库。
2. **进程管理**：使用 `spawn` 启动多进程，每个进程通过 `init_process_group` 初始化，配置 `MASTER_ADDR/PORT` 用于协调，实际数据传输由 NCCL 通过 NVLink/InfiniBand 完成。
3. **同步机制**：`barrier` 用于进程级同步，`torch.cuda.synchronize` 用于 CUDA 内核级同步。两者缺一不可，顺序错误会导致伪同步和数据竞争。
4. **三大核心操作**：
 - **All-Reduce**：就地求和并广播，是数据并行梯度同步的主力。

⁰ 视频截图来源：00:52:08。

- **Reduce-Scatter**: 输入输出分离，逐维度规约后分散结果，避免单点显存爆炸。
 - **All-Gather**: 收集分散的分片，让每个 rank 获得完整结果。
5. **核心等价关系**: $\text{All-Reduce} \equiv \text{Reduce-Scatter} + \text{All-Gather}$ 。在 ZeRO/FSDP 等高级技术中，显式拆分这一等式可以获得更大的优化灵活性。
 6. **异步通信**: 通过 `async_op=True` 实现计算与通信重叠，是提升分布式训练效率的关键技术。
 7. **带宽基准测试**: 严格遵循“预热 → 同步 → 计时 → 同步”的流程，计算有效带宽。All-Reduce 带宽 $\approx 2 \times \text{size_bytes} / \text{duration}$ ，与进程数和拓扑无关。

掌握了这些编程实践后，我们已经具备了实现和理解三种并行训练策略（数据并行、张量并行、流水线并行）的代码能力。下一节将把这些基础构建块组合起来，系统讲解如何在 PyTorch 中实现每一种并行策略，并分析其通信开销、适用场景与硬件要求。

5 三大并行训练策略

在掌握了集体通信原语与硬件互联的基础之后，我们进入本讲的核心主题：如何将一个庞大的语言模型训练任务拆分到多块 GPU 上执行。本节以深度 MLP 为示例（回顾前文，MLP 正是 Transformer 中的计算瓶颈），依次讲解三种最基础的并行策略——数据并行、张量并行与流水线并行。理解这三种策略的切分维度、通信模式与适用场景，是构建大规模分布式训练系统的起点。

5.1 数据并行 (DDP): 分数据、全参数

5.1.1 动机与核心思想

数据并行 (Data Parallelism, DP) 是最直观、最优雅的并行方式。其出发点非常简单：当单卡无法在一次前向/反向传播中处理完整的训练批次时，我们可以将全局批次 (global batch) 沿**样本维度**切分为若干局部批次 (local batch)，每个 GPU 只处理其中一部分数据，但各自维护一份完整的模型参数副本。

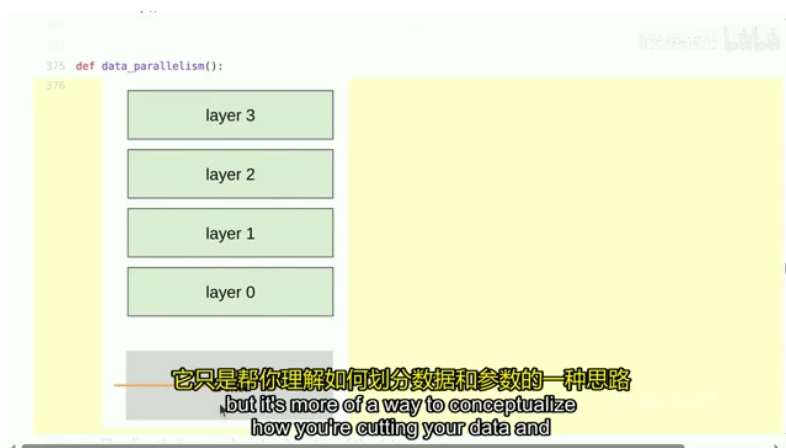


图 31: 数据并行的切分示意：每个 rank 获得数据的一个切片，但拥有完整的层结构。

⁰来源：讲座代码演示，约 00:55:15。

如图 31所示，数据并行的”分治”只发生在数据矩阵的行方向上；模型本身的层结构（layer 0 到 layer 3）在每个 rank 上都是完整的。这意味着前向传播与反向传播的计算逻辑几乎与单卡训练完全一致。

5.1.2 机制：局部批次与梯度同步

设全局批次大小为 B ，进程总数（world size）为 N ，则每个 rank 的局部批次大小为：

$$b = \frac{B}{N} \quad (8)$$

讲座中以 $B = 128$ 、 $N = 4$ 为例，每个 rank 处理 $b = 32$ 条样本。数据切分通过简单的索引操作完成：

```
local_batch_size = batch_size // world_size
start_index = rank * local_batch_size
end_index = start_index + local_batch_size
data = data[start_index:end_index].to(cuda_if_available(rank))
```

每个 rank 独立执行前向传播与反向传播。由于输入数据不同，各 rank 计算出的损失值和梯度自然也不同。数据并行能够正确工作的关键，在于反向传播后插入的**梯度同步**步骤——通过 All-Reduce 操作将所有 rank 上的梯度求平均，使得每个 rank 最终获得完全一致的梯度，从而保证参数更新的一致性。

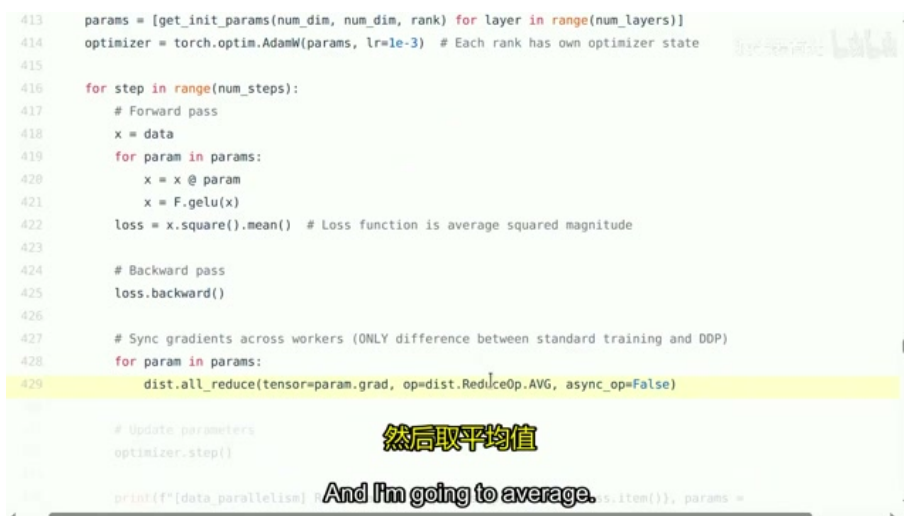


图 32: 数据并行训练循环的核心代码。唯一与标准训练的区别在于反向传播后的 All-Reduce 梯度同步。

如图 32所示，关键代码仅有一行：

```
dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG, async_op=False)
```

这行代码对所有 rank 上的梯度执行 All-Reduce 求平均。完成后，每个 rank 的 `param.grad` 完全相同，随后调用 `optimizer.step()` 即可更新参数。由于梯度一致，各 rank 的参数在每一步之后仍然保持同步。

⁰来源：讲座代码演示，约 00:59:20。

数据并行的本质

数据并行本质上仍是标准训练流程，只是将全局批次拆分到多个设备上并行处理。它与单卡训练的**唯一区别**在于反向传播后增加了一次 All-Reduce 梯度同步。这种模块化设计使得 DDP 对模型结构完全透明——无论前向传播是 MLP、Transformer 还是其他架构，DDP 只负责在参数层面做平均。

5.1.3 DDP 的优雅性与局限

DDP 的模块化优势体现在：你只需关注前向传播的逻辑，无需修改模型内部结构。对于 Transformer 基座模型，DDP 同样适用——它不关心你的模型具体是什么，只对所有参数的梯度执行统一的 All-Reduce 操作。

然而，DDP 有一个根本性的前提：**每个 rank 必须能够容纳完整的模型参数、梯度和优化器状态**。当模型规模增长到单卡显存无法承载时（例如 1 万亿参数模型的参数、梯度和 Adam 优化器状态远超 192 GB HBM 容量），DDP 便不再可行。此时需要更聪明的数据并行变体——FSDP (Fully Sharded Data Parallel) 或 ZeRO (Zero Redundancy Optimizer)，其核心思想是将参数、梯度和优化器状态也分片到各个 rank 上，通过 All-Gather 与 Reduce-Scatter 的组合来避免显存冗余。这些进阶内容将在后续课程中详细展开。

实践注意事项

- 全局批次大小 B 应至少等于 world size N ，最好是 N 的整数倍；若不能整除，可用零填充 (padding) 处理。
- 数据加载最好由每个 rank 独立完成，避免单点瓶颈。
- 各 rank 上的损失值不同（因为数据不同），但梯度经 All-Reduce 后一致，因此参数始终保持同步。

5.2 张量并行 (TP)：分参数、全数据

5.2.1 动机与核心思想

当模型宽度（即每层参数矩阵的维度）过大，单卡无法容纳完整的一层时，我们需要沿**张量维度**切分参数。张量并行 (Tensor Parallelism, TP) 的核心思想是：每个 rank 只拥有每层权重矩阵的一部分，但所有 rank 都接收**完整的数据批次**。这与数据并行恰好形成对偶——数据并行是“分数据、全参数”，张量并行是“分参数、全数据”。

如图 33 所示，张量并行沿水平方向（宽度维度）切分模型。每个 rank 仍然拥有完整的层数 (layer 0 到 layer 3)，但每一层的参数矩阵只有原来的 $1/N$ 。

5.2.2 机制：列式切分与 All-Gather 通信

讲座中以**列式张量并行** (column tensor parallelism) 为例。假设原始权重矩阵的维度为 $D \times D$ ，world size 为 N ，则每个 rank 只负责一个 $D \times d_{\text{local}}$ 的子矩阵，其中 $d_{\text{local}} = D/N$ 。

前向传播的执行流程如下：

⁰来源：讲座代码演示，约 01:03:00。

⁰来源：讲座代码演示，约 01:04:30。

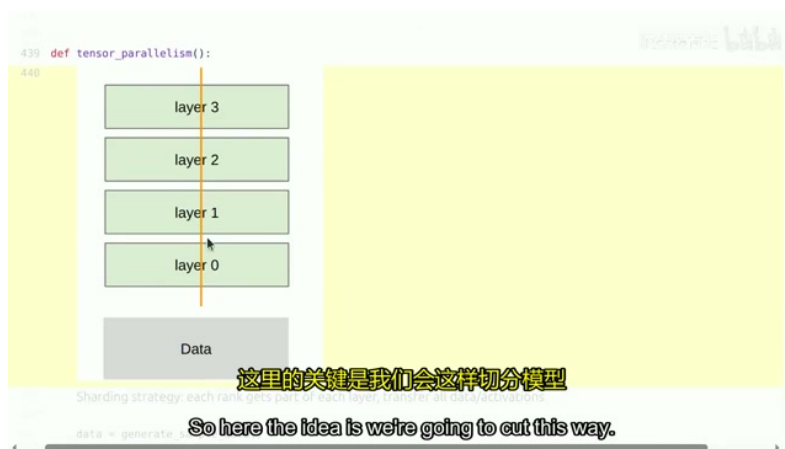


图 33: 张量并行的切分示意: 沿宽度维度 (列方向) 切分每层的参数矩阵, 每个 rank 获得每层的部分参数。

```

457 # W0 W1 W2 W3
458 # | | | |
459 params = [get_init_params(num_dim, local_num_dim, rank) for layer in range(num_layers)]
460
461 # Forward pass
462 x = data
463 for layer in range(num_layers):
464     # Compute activations (batch_size x local_num_dim)
465     x = x @ params[layer] # Note: this is only on a slice of the parameters
466     x = F.gelu(x)
467
468     # Allocate memory for activations (world_size x batch_size x local_num_dim)
469     activations = [torch.empty(batch_size, local_num_dim, device=cuda_if_available(rank)) for _ in range(w
470
471     # Send activations via all gather
472     dist.all_gather(tensor_list=activations, tensor=x, async_op=False)
473
474     # Concatenate them to get batch_size x num_dim
475     x = torch.cat(activations, dim=1)
476
477 print(f"[tensor_parallelism] Rank {rank}: forward pass produced activations {summarize_tensor(x)}", flush=

```

图 34: 张量并行的前向传播代码。每个 rank 只计算局部激活值, 然后通过 All-Gather 汇总完整激活值。

1. 每个 rank 接收完整数据 X (形状 $B \times D$)。
2. 计算局部激活值: $X_{\text{local}} = X \cdot W_{\text{local}}$, 形状为 $B \times d_{\text{local}}$ 。
3. 应用逐元素非线性激活 (如 GELU), 这一步无需通信。
4. 通过 **All-Gather** 将所有 rank 的局部激活值拼接为完整激活值 X' (形状 $B \times D$), 供下一层使用。

对应的代码逻辑为:

```
# 计算局部激活值 (batch_size x local_num_dim)
x = x @ params[layer] # 仅使用参数的一部分
x = F.gelu(x)

# 通过All-Gather收集所有rank的激活值
activations = [torch.empty(batch_size, local_num_dim, ...)
                 for _ in range(world_size)]
dist.all_gather(tensor_list=activations, tensor=x, async_op=False)

# 拼接得到完整激活值 (batch_size x num_dim)
x = torch.cat(activations, dim=1)
```

5.2.3 反向传播的对偶性

张量并行不仅改变了前向传播, 也深刻影响了反向传播。在前向传播中, 我们使用了 All-Gather 来汇总激活值; 根据集体通信的对偶性, 在反向传播中对应地需要使用 **Reduce-Scatter** 来分散梯度。



图 35: 讲座中讲解反向传播时的 Reduce-Scatter 对偶性。

具体来说:

- 前向: All-Gather 将分散在各 rank 的局部激活值收集为完整激活值。
- 反向: Reduce-Scatter 将完整梯度规约后分散回各 rank, 使每个 rank 只获得对应局部参数维度的梯度。

⁰来源: 讲座现场, 约 01:08:15。

手动管理 vs. 自动封装

在 PyTorch 中，如果你直接调用 `backward()`，它不会自动执行这些并行的 Reduce-Scatter 操作——普通的反向传播没有内置并行通信机制。不过 PyTorch 的高层 API（如 `torch.distributed.tensor`）已经将这些操作封装好，可以自动完成。本课程采用显式方式管理这些操作，目的是让读者从底层机制上理解分布式训练的本质。

5.2.4 张量并行的通信代价

与数据并行相比，张量并行有一个显著区别：**每一层前向传播都需要通信**。数据并行只在反向传播结束时执行一次 All-Reduce；而张量并行在每一层的前向传播后都要执行 All-Gather，反向传播中还要执行 Reduce-Scatter。这意味着张量并行的通信频率与模型深度成正比。

更关键的是，All-Gather 传输的是**激活值**，其数据量通常非常大（批次大小 $\times$ 维度数）。因此，张量并行对互联带宽的要求极高——通常只适用于节点内部通过 NVLink/NVSwitch 连接的高带宽环境。一旦跨出 NVLink 域（例如跨节点通过 InfiniBand 或以太网连接），张量并行的通信开销将迅速成为瓶颈。

5.3 流水线并行 (PP): 分层切分与微批次

5.3.1 动机与核心思想

流水线并行 (Pipeline Parallelism, PP) 采取第三种切分维度：沿**网络深度**（层方向）将模型切分为若干阶段 (stage)，每个 rank 负责一个连续的层块。与数据并行和张量并行不同，流水线并行中每个 rank 拥有其负责层的**全部参数维度**，但只拥有**部分层数**。



图 36: 流水线并行的核心概念：沿深度方向切分网络，每个 rank 负责一部分连续层。

这种切分方式非常自然：深度神经网络本身就是一层一层顺序计算的，将连续的若干层分配给同一个 GPU，然后在 GPU 之间传递中间激活值，完全符合直觉。

5.3.2 机制：点对点通信与微批次

在流水线并行中，每个 rank 以**点对点** (point-to-point) 方式与相邻 rank 通信：从前一个 rank **接收** 输入激活值，对本地的层执行前向传播，再**发送** 输出激活值给下一个 rank。

⁰ 来源：讲座现场，约 01:09:45。

```

# 从前一个rank接收数据
if rank > 0:
    x = recv_tensor_from_rank(rank - 1)

# 对本地层执行前向传播
for layer in local_layers:
    x = x @ params[layer]
    x = F.gelu(x)

# 发送给下一个rank
if rank < world_size - 1:
    send_tensor_to_rank(rank + 1, x)

```

然而，如果简单地一次处理整个全局批次，流水线并行会产生严重的**流水线气泡**（pipeline bubble）——即某些 rank 在空闲等待上下游数据的时间段。如图 37所示，当一个 rank 处理完自己的层后，必须等待后续 rank 完成计算并传回梯度，这个等待期间 GPU 处于空闲状态。



图 37: 讲座中讲解微批次如何减少流水线气泡。

5.3.3 微批次：减少流水线气泡

解决流水线气泡的核心思想是引入**微批次**（micro-batch）：将全局批次进一步细分为更小的子批次。每个微批次可以独立地流经整个流水线。当一个 rank 处理完第一个微批次并将其发送给下一个 rank 后，它可以立即开始处理第二个微批次，而不必等待第一个微批次的反向传播完成。

这种“流水化”的执行方式大大减少了空闲等待时间。微批次越小，流水线填充得越满，气泡占比越低。但微批次过小也会带来额外开销（更多的通信次数和更小的计算粒度），因此需要在实践中权衡。

5.3.4 通信与计算重叠

讲座特别强调了另一个优化方向：**通信与计算重叠**（communication/computation overlap）。在基础版本中，通信操作是同步阻塞的；但通过使用异步通信原语（如 `async_op=True`），可以在发送/接收数据的同时执行本地计算。

⁰来源：讲座现场，约 01:12:30。

这一优化在流水线并行中尤为关键：一个 rank 可以在计算当前微批次的同时，异步接收上一个微批次来自前一个 rank 的梯度，或发送当前微批次的激活值给下一个 rank。类似地，在数据并行的反向传播中，一旦某层梯度计算完成，就可以立即开始 All-Reduce 同步，而不必等到所有层的梯度都算完。这种细粒度的重叠能显著减少实际等待时间。

5.4 三种并行策略的对比与选择

5.4.1 综合对比

表 2 从切分维度、通信模式、硬件要求和适用场景四个角度，对三种并行策略进行了系统对比。

表 2: 三种基础并行训练策略对比

维度	数据并行 (DDP)	张量并行 (TP)	流水线并行 (PP)
切分对象	数据批次 (行)	参数矩阵 (列/行)	网络层 (深度)
每个 rank 拥有	完整参数	部分参数	部分层 (完整维度)
每个 rank 处理	部分数据	全部数据	全部数据
通信原语	All-Reduce (梯度)	All-Gather + Reduce-Scatter	Send/Recv (点对点)
通信频率	每步一次	每层一次	每阶段一次
通信数据量	梯度 (与参数同量)	激活值 (大)	激活值 (中等)
带宽要求	中等	极高 (NVLink 域内)	较低 (可跨慢速网络)
模型透明度	完全透明	需修改模型结构	需修改模型结构
显存瓶颈	完整参数副本	参数分片	层分片

5.4.2 硬件驱动的策略选择

选择哪种并行策略，很大程度上取决于你的硬件拓扑：

- **张量并行**需要极高的互联带宽,因为每层都要传输大量激活值。因此它通常被限制在单个 NVLink 域内 (如 NVL72 的 72 块 GPU 之间)。跨 NVLink 域使用张量并行效率极低。
- **流水线并行**对带宽要求低得多，能够适应慢速互联 (如跨地域的以太网)。因此一些去中心化训练场景会采用流水线并行——即使 GPU 分布在半个地球之外，流水线并行仍然可以工作。
- **数据并行**的带宽需求介于两者之间。它可以扩展到相当大的规模,但会遇到**临界批次大小** (critical batch size) 问题：当批次大小超过临界值后，继续增大并不能带来训练效率的提升，只是在浪费算力。

5.4.3 混合同并行策略

在实际的大规模训练系统中，几乎从不单独使用某一种并行策略，而是将它们组合起来形成**混合同并行** (hybrid parallelism)。最常见的组合方式是：

1. **节点内部**：使用张量并行 (充分利用 NVLink 高带宽)。
2. **节点之间**：使用数据并行或 FSDP (扩展性强，通信频率低)。
3. **跨集群/广域网**：如需进一步扩展，可叠加流水线并行 (适应慢速互联)。

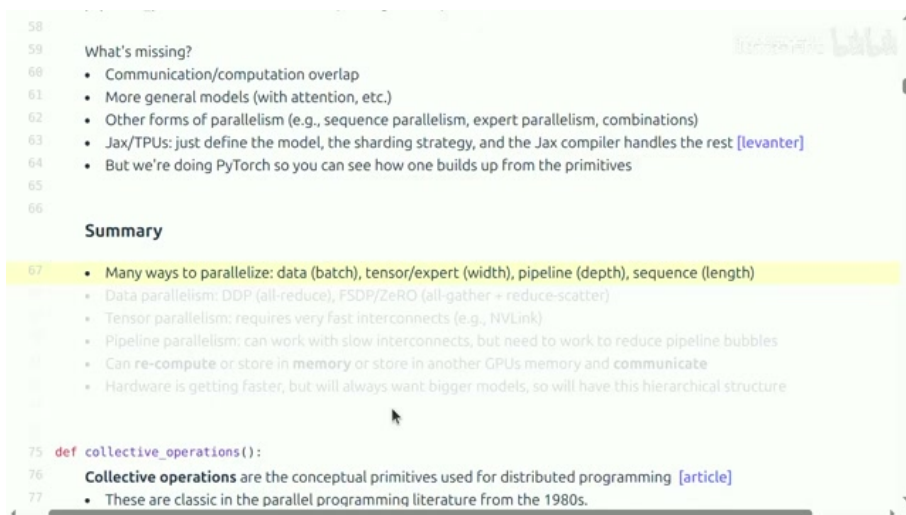


图 38: 讲座总结幻灯片：三种并行策略的切分维度与关键特性。

如图 38 所示，讲座将各种并行方式归纳为对不同维度的切分：数据 (batch)、张量/专家 (width)、流水线 (depth)、序列 (length)。这种多维度的切分思维，是设计大规模分布式训练系统的核心框架。

5.5 进阶方向与其他并行类型

除了本节介绍的三大基础策略，现代大模型训练还涉及多种进阶并行技术：

- **序列并行 (Sequence Parallelism)**：将长序列沿长度维度切分为多个片段，使注意力计算可以并行执行。这在处理超长上下文（如百万 token 级别）时尤为重要。
- **专家并行 (Expert Parallelism)**：用于 MoE (Mixture of Experts) 模型，将不同的专家模块分配到不同 GPU 上。这正是 All-to-All 集体通信发挥作用的典型场景——每个 token 需要被路由到对应的专家，计算完成后再汇总结果。
- **FSDP/ZeRO**：通过将参数、梯度和优化器状态分片到各 rank，在数据并行的基础上进一步降低单卡显存占用。其核心通信模式是 All-Gather (收集参数用于计算) 与 Reduce-Scatter (分散梯度用于更新) 的组合。

从底层原语到高层抽象

本课程刻意使用 PyTorch 最底层的集体通信 API (`dist.all_reduce`、`dist.all_gather` 等)，目的是让读者从机制层面理解分布式训练的本质。相比之下，JAX/TPU 等框架允许用户只定义模型和切分策略，编译器会自动处理通信操作的选择与调度。这种高层抽象虽然便捷，但会掩盖底层数据流动的细节。掌握原语，才能在面对新硬件、新拓扑时做出正确的并行决策。

5.6 本章小结

本章系统讲解了三种基础并行训练策略：

⁰来源：讲座总结幻灯片，约 01:19:10。

⁰来源：讲座现场，约 01:15:40。

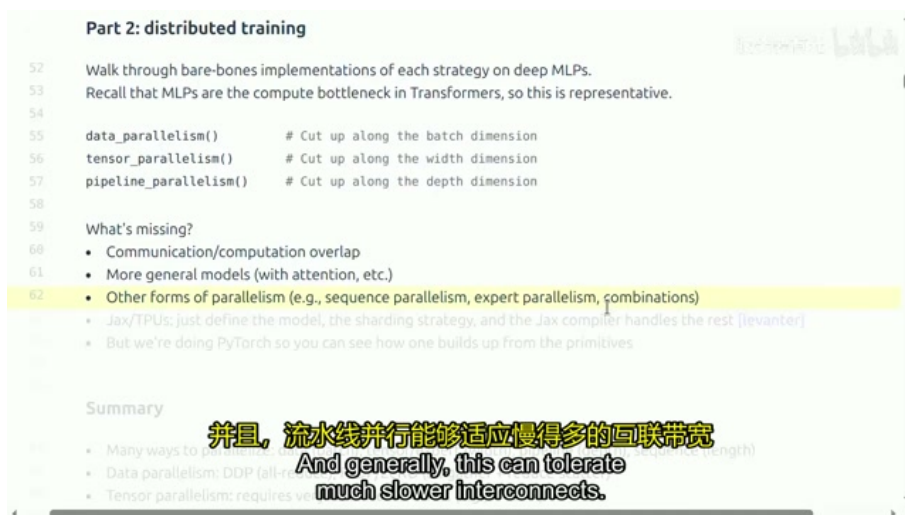


图 39: 讲座中提及专家并行与序列并行等其他并行类型。

1. **数据并行 (DDP)** 将全局批次沿样本维度切分，每个 rank 处理局部数据但维护完整参数。反向传播后通过 All-Reduce 同步梯度，确保参数一致性。其最大优势是模块化设计——对模型结构完全透明，只需改动一行代码。但前提是单卡能容纳完整模型。
2. **张量并行 (TP)** 将每层参数矩阵沿宽度维度切分，每个 rank 只拥有部分参数但处理全部数据。前向传播通过 All-Gather 汇总激活值，反向传播通过 Reduce-Scatter 分散梯度。由于每层都需要通信且传输的是大体积激活值，TP 对互联带宽要求极高，通常限制在 NVLink 节点内。
3. **流水线并行 (PP)** 将网络沿深度方向切分为若干阶段，每个 rank 负责一个连续层块。通过微批次 (micro-batch) 将全局批次进一步细分，以流水化方式减少流水线气泡。PP 使用点对点 Send/Recv 通信，对带宽要求较低，可适应慢速互联甚至广域网环境。

三种策略并非互斥，而是互补。实际的大规模训练系统通常采用混合并行：节点内张量并行 + 节点间数据并行/FSDP + 必要时叠加流水线并行。选择哪种组合，取决于硬件拓扑 (NVLink 带宽 vs. 跨节点带宽)、模型规模 (宽度瓶颈 vs. 深度瓶颈) 和训练配置 (批次大小、序列长度等)。

此外，序列并行、专家并行、FSDP/ZeRO 等进阶技术进一步扩展了并行的维度。随着硬件性能不断提升和模型规模持续增长，这种**分层、多维度的并行切分思路**将长期主导大规模深度学习训练系统的设计。

6 总结与延伸

6.1 讲座核心脉络回顾

本讲围绕”如何利用多块 GPU 加速语言模型训练”这一核心问题，从三个层面展开了系统讲解：

1. **通信原语层面**：我们系统学习了 8 种集体通信操作 (Broadcast、Scatter、Gather、Reduce、All-Gather、Reduce-Scatter、All-Reduce、All-to-All)，并重点掌握了训练中最核心的 All-Reduce 及其等价分解 (Reduce-Scatter + All-Gather)。这些原语是分布式训练的”汇编语言”，所有高层并行策略最终都映射到这组操作上。

2. **硬件拓扑层面**：我们理解了从 PCIe 到 NVLink/NVSwitch 再到 InfiniBand 的带宽递减规律，以及 RDMA 技术如何绕过 CPU 实现 GPU 间直接内存访问。NCCL 库作为硬件与软件之间的桥梁，自动将高层集体操作映射到最优的物理路径上。
3. **编程实践层面**：通过 PyTorch 的 `torch.distributed` 库，我们亲手实现了 All-Reduce、Reduce-Scatter 和 All-Gather，并学会了用 `async_op=True` 重叠计算与通信，以及如何准确测量有效带宽。
4. **并行策略层面**：我们对比了数据并行（DDP）、张量并行（TP）和流水线并行（PP）三种基础策略的切分维度、通信模式和适用边界，并理解了实际系统中混合并行的必要性。

6.2 核心洞察与实用建议

从单卡到集群的统一优化思想

无论是优化单 GPU 内核还是设计多 GPU 分布式训练系统，核心问题始终是同一个：**计算单元离数据有多远？**在单 GPU 中，答案由 HBM 带宽决定；在多 GPU 中，答案由 NVLink、InfiniBand 或以太网的带宽决定。因此，所有并行策略的设计目标都可以归结为一句话：**最大化计算时间占比，最小化通信时间占比。**

对于实际工程选择，可以遵循以下决策流程：

1. **模型能否装入单卡？**如果能，优先使用**数据并行（DDP）**——它对模型结构完全透明，改动最小，收益最稳定。
2. **模型宽度是瓶颈？**如果单层参数量过大（如超大嵌入表或宽 MLP），考虑**张量并行（TP）**——但需确保 TP 限制在 NVLink 全互联的节点内部，否则通信开销会急剧上升。
3. **模型深度是瓶颈？**如果层数极多（如数百层的 Transformer），考虑**流水线并行（PP）**——它对带宽要求低，适合跨节点部署，但需要精心设计微批次数量以平衡流水线气泡和内存占用。
4. **显存还是不够？**当上述策略仍无法解决显存压力时，引入 **FSDP/ZeRO**——将优化器状态、梯度甚至参数本身分片到所有 rank 上，以通信换显存。

6.3 前沿方向与开放问题

进阶并行技术

- **序列并行 (Sequence Parallelism)**: 当序列长度极长时 (如 1M token), 即使 batch size 很小, 激活值的内存占用也可能超出单卡容量。序列并行将序列维度切分到多个 GPU 上, 与 TP/PP 正交组合, 可进一步扩展上下文长度。
- **专家并行 (Expert Parallelism)**: 在混合专家模型 (MoE) 中, 不同的 token 被路由到不同的专家子网络。专家并行将不同专家放置在不同 GPU 上, 通过 All-to-All 操作实现动态路由, 是 GPT-4、Mixtral 等模型的关键扩展技术。
- **上下文并行 (Context Parallelism)**: 针对超长上下文 (如 1M+ token) 的专用并行维度, 将注意力计算沿序列长度切分, 配合环形注意力 (Ring Attention) 等技术减少通信量。
- **自动并行 (Auto-Parallelism)**: Alpa、Megatron-LM、DeepSpeed 等框架尝试自动搜索最优的并行策略组合, 将人工设计的启发式规则转化为优化问题求解。

常见陷阱

- **忽视通信开销**: 简单认为“GPU 数量翻倍, 训练速度也翻倍”是危险的。如果通信时间占比过高 (如跨慢速以太网执行 TP), 增加 GPU 反而可能降低整体效率。
- **混淆并行维度**: TP 切分宽度、PP 切分深度、DP 切分数据——三种维度彼此正交, 但它们的组合并非任意。错误的组合 (如在慢速网络上强推 TP) 会导致灾难性的性能下降。
- **低估同步开销**: Barrier、All-Reduce 等同步操作会强制所有 rank 等待最慢者。如果负载不均衡 (如某些 GPU 处理更长序列), 同步开销会被放大到最坏情况。

6.4 延伸资源

- NVIDIA NCCL 官方文档: <https://docs.nvidia.com/deeplearning/nccl/>
- PyTorch Distributed 教程: https://pytorch.org/tutorials/beginner/dist_overview.html
- Megatron-LM 论文 (张量并行原始工作): *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*
- DeepSpeed ZeRO 论文: *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*
- GPipe 论文 (流水线并行): *GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism*